

UNIVERSITÀ DEGLI STUDI DI SALERNO

Department of Computer Science

Bachelor's Degree in Computer Science

BACHELOR'S THESIS

Identifying and correcting prompt smells when employing LLMs in software development processes

SUPERVISORS

Prof. Fabio Palomba

Dott. Antonio della Porta

Dott. Gianmario Voria

Università degli Studi di Salerno

CANDIDATE

Luca Salvatore

Matricola: 0512119210

Academic Year 2025-2026

This thesis was carried out at



I am not a perfectionist, but I like to feel that things are done well. More important than that, I feel an endless need to learn, to improve, to evolve to feel satisfied with myself.

~Cristiano Ronaldo

Abstract

Nowadays, an increasing number of people rely on Large Language Models on a daily basis to generate or refine text, code, and e-mails, or simply to ask questions. A natural consequence of the rapid diffusion of these models has been the emergence of Prompt Engineering, a research field concerned with identifying novel techniques that, when applied by the user at the time of prompt formulation, can improve the quality of an LLM's inherently non-deterministic response. However, very few researchers have focused on treating the prompt itself as an artifact whose quality can be assessed independently of the output it elicits.

Prompt smells, a notion borrowed from code smells, are weaknesses that do not make a prompt wrong but tend to accompany poorer output. The only existing catalog of prompt smells in literature rests on a rapid literature review, was never tested on large amounts of data, and says nothing about whether the smells can be fixed. The purpose of this thesis is to address these three gaps.

A tertiary study of systematic reviews on prompt engineering in software development produced eleven recurring bad practices, of which only five were considered to be associated with a measurable metric. We measured these five, together with other five smells of the existing catalog, on 42,140 real developer-LLM prompts. The experiment revealed a high prevalence of prompt smells in the dataset, especially of the first group, and that the smells are largely independent of one another. Mitigation turned out to be feasible for three smells only.

The detection and correction algorithms used to carry out the experiments were released as an open-source tool, *NasoMano*, available through a REST API embedded in a Docker image and connected to a minimal graphical user interface.

Contents

List of Figures **iv**

List of Tables **v**

1 Introduction **1**

1.1 Context 1

1.2 Motivation and Objectives 2

1.3 Results 3

1.4 Thesis Structure and Workflow 4

2 Background and Related Work **6**

2.1 Large Language Models 6

2.1.1 The concept of inference 7

2.1.2 LLMs for code-related tasks 8

2.2 Prompt Engineering 9

2.3 Prompt Smells 10

2.4 Research Gaps 12

3 Research Method **14**

3.1 Research Questions 14

3.2 Tertiary Review 15

3.2.1 Planning Phase . . . 16
3.2.2 Conducting Phase . . . 17
3.3 Large-scale Evaluation . . . 22
3.3.1 Dataset Selection . . . 22
3.3.2 Which Prompt Smells Can Be Detected, And How? . . . 24
3.3.3 Regular Expressions Baseline . . . 26
3.3.4 LLM-as-a-Judge Approach . . . 27
3.4 Smells Mitigation . . . 32
3.4.1 How are smells corrected? . . . 32
3.4.2 Why aren't the other seven smells corrected? . . . 34

4 Analysis of the Results **36**
4.1 RQ1: A Systematic Catalog Of Prompt Smells . . . 36
4.1.1 Ambiguity and Challenges identified . . . 38
4.2 RQ2: On Large-Scale Assessment of Prompt Smells . . . 39
4.2.1 Application to a Large-Scale Dataset . . . 39
4.2.2 Correlation Between Prompt Smells . . . 41
4.3 RQ3: Empirical Validation of Smells Mitigation . . . 42
4.3.1 Large-Scale Correction and Re-evaluation . . . 43
4.3.2 Relation Between False Positives and Prompt Length . . . 43

5 The Development of NasoMano **46**
5.1 Naso - prompt smells detection tool . . . 46
5.1.1 Syntactic Smells Detection . . . 47
5.1.2 LLM-as-a-judge Inference Configuration . . . 48
5.2 The NasoMano API . . . 49
5.3 NasoMano Class Diagram . . . 52
5.4 Containerization of the API . . . 52
5.5 Graphical User Interface . . . 53

6 Conclusions **55**
6.1 Summary of the Thesis . . . 55
6.2 Main Findings . . . 56

6.3 Future Developments 57

Bibliography **59**

List of Figures

1.1 The workflow employed for this thesis project 5

3.1 Procedure for obtaining the final dataset 23
3.2 Line plot of percentages of prompt smells detected in the experiments 31
3.3 The final version of the instructions to be given to the LLM judge . . 33

4.1 Detection rate for each prompt smell 40
4.2 Correlation matrix between prompt smells 41
4.3 Estimated probability of false positives as a function of length 44

5.1 Ollama client configuration 48
5.2 Pydantic model used for guided decoding 49
5.3 UML sequence diagram for the `/api/detect` endpoint 50
5.4 UML sequence diagram for the `/api/fix` endpoint 51
5.5 UML sequence diagram for the `/api/refactor` endpoint 51
5.6 UML class diagram of the *NasoMano* tool 52

List of Tables

2.1 Della Porta et al.'s catalog of prompt smells and metrics 12

3.1 Inclusion and Exclusion Criteria. 17
3.2 Cases of disagreement during the preliminary screening phase 19
3.3 Articles taken in account during the first backward snowballing iteration 20
3.4 Final Articles from which data were extracted 21
3.5 Percentages of prompt smells detected in multiple experiments . . . 30

4.1 A first, unpolished catalog of bad practices in prompt writing 37
4.2 The final systematic prompt smells catalog 37
4.3 Count of prompt smells detected in the dataset 40

5.1 List of prompt smells' metrics that the tool *Naso* can compute 47

CHAPTER 1

Introduction

1.1 Context

Large language models (LLMs) are taking on an increasingly important role in software engineering workflows, supporting tasks such as code generation, code completion, and debugging via natural language prompts rather than explicit programming. As the use of these models has increased, prompt engineering, the practice of designing and formulating prompts to maximize the quality of an LLM's output, has also assumed a fundamental role. The literature contains numerous techniques for creating effective prompts, such as thought-chain reasoning, role assignment, and few-shot examples, the aim of which is to reduce ambiguity and improve the reliability of the generated code. At the same time, an inadequately worded prompt can have the opposite effect, increasing the likelihood of inaccurate or unreliable output. This is precisely where the concept of prompt smells comes from, which draws an explicit analogy with code smells in software engineering. This represents a superficial indicator of a weakness within the prompt that, while not a real error, is related to compromised output quality.

1.2 Motivation and Objectives

The scientific literature contains a significant body of work on good prompt engineering practices, but comparatively little systematic attention has been paid to cataloging bad practices, i.e., "prompt smells", and to associating them with measurable metrics. Existing taxonomies also tend to focus on prompt engineering in general rather than specifically on prompt engineering for software development or code generation.

A study from Della Porta et al. [1] is among the first attempts to treat the prompt as an artifact whose quality can be assessed independently of the output it elicits. However, it is explicitly presented as a preliminary study and leaves several gaps open. First, its catalog of prompt smells is not systematic, as it derives from a rapid rather than a systematic literature review. Second, its empirical evidence lacks large-scale analysis, so it remains unclear whether the findings hold across larger and more heterogeneous corpora of prompts. Third, the effect and mitigability of prompt smells are unknown: no mitigation is applied to smelly prompts, so we do not know whether the smells can be removed without altering the intent of the prompt, nor whether their removal yields measurable improvements.

This thesis originates from the need to address these three gaps. Rather than focusing on best practices, it investigates which prompt bad practices or anti-patterns have been identified in the literature and whether each can be associated with a metric suitable for automatic detection. It then examines how prevalent these smells are in real-world developer–LLM conversations, and to what extent they can be corrected automatically. Answering these questions is a necessary step toward the automated assessment and improvement of prompt quality in software development.

◎ **Our Goal.** This thesis aims to systematically catalog prompt smells in software development together with their evaluation metrics, to measure their prevalence on a large-scale dataset of real-world prompts, and to assess to what extent they can be automatically mitigated.

1.3 Results

The results show that prompt smells can be systematically cataloged and measured, that they are highly prevalent in real-world developer-LLM prompts, and that they can be mitigated only partially.

For the catalog, the tertiary review identified eleven recurring bad practices in prompt writing. Only five of them could be associated with a meaningful evaluation metric: *Reasoning Suppression*, *Lack of Self-Reflection*, *Role Suppression*, *Unspecified Output Structure*, and *Lack of Examples*. All five correspond to the absence of an established prompt pattern, so their metrics are binary, except for *Lack of Examples*, which is measured as a count.

At the large scale, these five smells were detected in 79.3% to 99.0% of the 42,140 analyzed prompts, depending on the smell. The syntactic smells taken from the catalog of Della Porta et al. are less widespread: four of the five affect between 58.6% and 65.2% of the prompts, while *Complexity-Length* affects only 7.1%, which suggests that its threshold or formula should be reconsidered. Most smells are largely independent of one another. The only strong association is between *Reasoning Suppression* and *Lack of Self-Reflection* ($r = 0.50$).

For mitigation, the results are more limited. A correction strategy based on injecting a short, pattern-compliant sentence was feasible only for *Role Suppression*, *Reasoning Suppression*, and *Lack of Self-Reflectio*n. After correction, each of these smells was still reported in a small share of prompts (between 1.87% and 3.91%). The probability of such residual detections increases with prompt length, and among the longest prompts, those above the 99th percentile, the rate rises to between 41.9% and 53.6%.

Finally, the detection and correction algorithms were released as an open-source tool, *NasoMano*, that can be used through a REST API embedded in a Docker image and connected to a minimal graphical user interface. The link to the project repository is: `https://github.com/lucasalvaa/NasoMano`.

1.4 Thesis Structure and Workflow

The remainder of this thesis is organized as follows:

- **Chapter 2 – Background and Related Work** introduces the foundational concepts needed to understand the contribution of this work. It first provides an overview of Large Language Models and the variants specialized in code-related tasks (Section 2.1). It then presents the main prompt engineering techniques proposed in the literature (Section 2.2) and reviews the state of the art on prompt smells, with particular attention to the catalog proposed by Della Porta et al. (Section 2.3). The chapter concludes by identifying the three research gaps that motivate this thesis (Section 2.4).

- **Chapter 3 – Research Method** describes the methodology adopted to address these gaps. It formulates one research question for each gap (Section 3.1) and then details the three phases of the study. The first phase is the tertiary review used to build a systematic catalog of prompt smells (Section 3.2). The second is the large-scale evaluation, which covers the dataset selection and experiments two detection approaches, based on regular expressions and on an LLM-as-a-judge (Section 3.3). The third is the mitigation strategy devised for a subset of the cataloged smells, together with the reasons why the remaining smells were not corrected (Section 3.4).

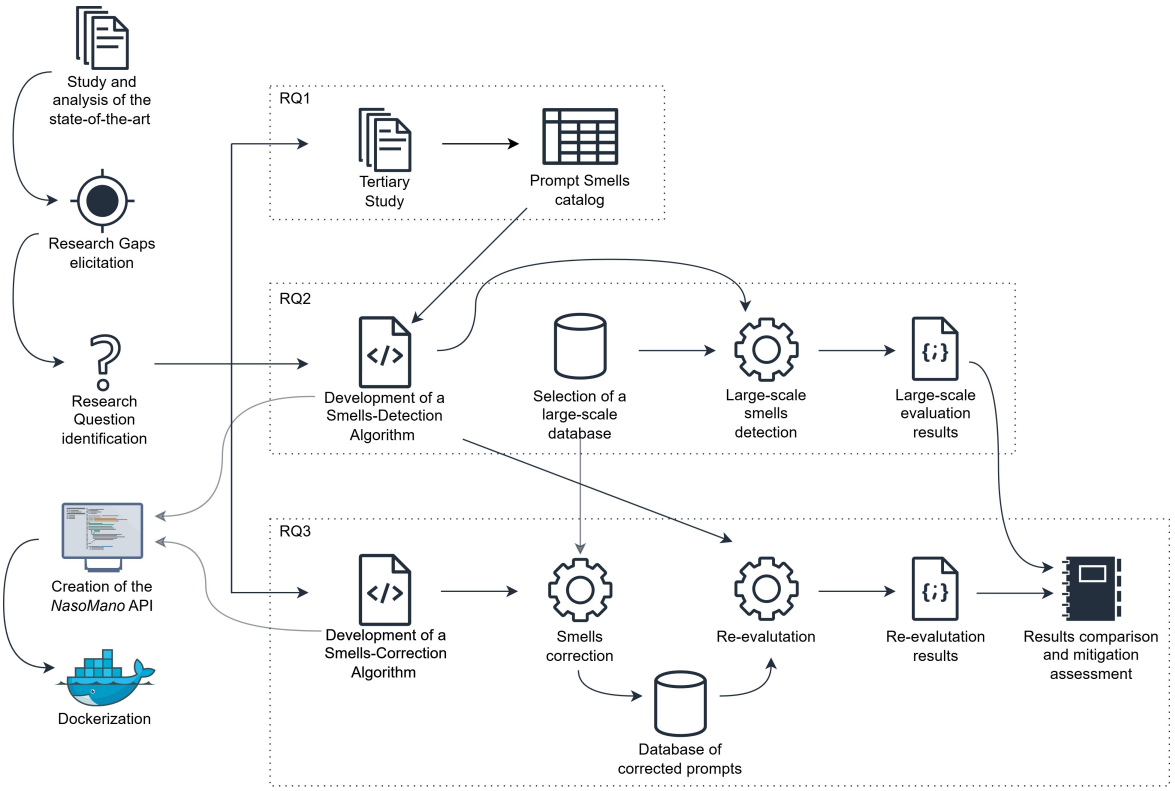
- **Chapter 4 – Analysis of the Results** presents the findings for each research question. It reports the systematic catalog of prompt smells obtained from the tertiary review, along with the ambiguities and challenges identified during data extraction (Section 4.1). It then analyzes the prevalence of the smells and the correlations among them on the large-scale dataset (Section 4.2). Finally, it assesses the effectiveness of the mitigation strategy, including the relationship between false positives and prompt length (Section 4.3).

- **Chapter 5 – The Development of NasoMano** illustrates how the detection and correction algorithms were packaged into an open-source tool composed of two independent components, *Naso* and *Mano*. This chapter describes the libraries

used to implement the detection component (Section 5.1), the REST API that exposes its functionality (Section 5.2), the class diagram of the tool (Section 5.3), its deployment with Docker and the rationale behind this choice (Section 5.4), and the GUI built to make the tool accessible to end users (Section 5.5).

- **Chapter 6 – Conclusions** summarizes the thesis (Section 6.1), discusses the main findings with respect to each research question (Section 6.2), and outlines directions for future research (Section 6.3).

Figure 1.1: The workflow employed for this thesis project



CHAPTER 2

Background and Related Work

This chapter introduces the foundational concepts necessary to understand the contribution of this thesis, covering Large Language Models in general and their variants specialized in code generation (Section 2.1), a brief overview of Prompt Engineering (Section 2.2); it then provides an overview of the current state of the art of the emerging concept of *Prompt Smells* (Section 2.3), highlighting the current research gaps that this thesis aims to address (Section 2.4).

2.1 Large Language Models

Large Language Models (LLMs) are machine learning models trained on a vast amount of text to understand, summarize, translate and generate text sequences, including source code based on textual inputs called *prompts*. Before LLMs became what we know them as today, sequence modeling in natural language processing relied primarily on recurrent architectures, such as Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks. These architectures process a sequence of words one element at a time and therefore struggle to capture dependencies between distant words.

The introduction of the Transformer architecture in the groundbreaking article

Attention is all you need by Vaswani et al. [2] marked a turning point in the history of natural language processing. By relying entirely on a self-attention mechanism rather than recursion, the Transformer enables the model to directly relate any two parts of a sequence regardless of their distance, whilst allowing full parallelization during training. This change has proven to be so significant that basically all large-scale language models developed since then are based on the Transformer architecture or a variant thereof.

In order to accomplish its job, a large language model breaks down the input prompt into basic units of text—such as a word, part of a word, or punctuation mark—called tokens. In the second phase, the model generates the code in an auto-regressive manner, predicting the next token based on the initial prompt and the previously predicted tokens, until the task is completed. Therefore, the design of the prompt is directly related to the quality of the model's output.

2.1.1 The concept of inference

Once the model training is complete, putting it into practice involves performing inference: in other words, providing it with a prompt and letting it generate an output, whether that be text, code or something entirely different.

Hosted inference is by far the most common for general-purpose LLMs: the model runs on the provider's own infrastructure and the user simply sends a request and receives a response, typically via a chatbot interface or an API, without ever coming into contact with the underlying hardware. For instance, the models in the GPT family (e.g., GPT-5.6) are probably the most well-known examples of hosted inference models. More specifically, they are hosted on the OpenAI's servers, while ChatGPT or the OpenAI API are the interfaces through which users can use them.

The second option, local inference, is available only for models whose weights have been made public. In this case, the model is downloaded and run directly on the user's computer, which requires sufficient memory and, usually, a powerful GPU to store and process the model's parameters. As this can be prohibitive for larger models, techniques such as quantization are often used to reduce a model's memory footprint by storing its weights with reduced precision, thereby making it feasible to

run otherwise resource-intensive models on standard consumer hardware.

2.1.2 LLMs for code-related tasks

While general-purpose LLMs (e.g., GPT-5, Gemini 3.6 Flash) are trained on broad corpora spanning natural language and code, other models are specifically adapted to understand and generate source code, through a process known as fine-tuning. Fine-tuning consists of taking a model that has already been trained on a broad, general-purpose corpus, and continuing its training on a narrower, domain-specific dataset. Rather than learning from scratch, the model adjusts its existing internal parameters to better capture the statistical and structural regularities of the new domain, while retaining much of the general language understanding acquired during the original training phase. An example of this approach is a cloud-based model called *Codex*, introduced by Chen et al. [3]: it was obtained by fine-tuning GPT-3 on a large corpus of publicly available source code, allowing it to substantially outperform its general-purpose counterpart on code-synthesis tasks despite the two models sharing the same underlying architecture.

CodeLlama, introduced by Rozière et al. [4], further extends this idea, applying fine-tuning not as a single step but as a sequence of specialization stages, each intended to add a specific capability on top of a general-purpose foundation model, i.e., *Llama 2*. One of these stages introduces infilling: rather than only predicting the next token based on preceding text, as in standard auto-regressive generation, the model is additionally trained to predict a missing span of code given both the text that precedes it and the text that follows it. This changes the underlying prediction task from a purely left-to-right one to one where the model must reconcile information from both directions, which is intended to better reflect how developers actually write and edit code. This distinction is relevant in the field of code generation research, as it can affect how consistently a model responds to prompt quality — a factor that becomes particularly important when the input prompts are drawn from real-world datasets rather than curated examples.

2.2 Prompt Engineering

Prompt engineering is the study of techniques for designing and formulating effective prompts to maximize the quality of LLM output. Various techniques have been proposed in the literature, e.g., Chain-of-Thought prompting or the use of explicit context instructions. They aim to reduce ambiguity and ensure that there is sufficient information to produce accurate output for the user.

- **Chain-of-Thought (CoT)** prompting, introduced by Wei et al. [5], encourages the model to generate intermediate reasoning steps before producing a final answer, rather than jumping directly to a conclusion. This technique has been shown to improve performance on tasks requiring multi-step reasoning, including certain code generation and debugging tasks, by making the model's inference process more explicit and less prone to logical gaps.

- **Structured Chain-of-Thought (SCoT)**, introduced by Li et al. [6], extends the CoT paradigm by asking the model to first outline an intermediate structure — such as pseudo-code, program logic, or sequence/branch/loop constructs — before generating the final code. This is particularly relevant to code generation, as it aligns the reasoning step with programming constructs rather than free-form natural language reasoning, which has been reported to improve the structural correctness of generated code.

- **Logical Chain-of-Thought (LogiCoT)** prompting, introduced by Zhao et al. [7], extends the CoT paradigm by incorporating principles from symbolic logic to improve the reliability of the reasoning process. Rather than accepting each generated reasoning step at face value, LogiCoT applies a verification mechanism based on *reductio ad absurdum*, allowing incorrect intermediate steps to be identified and revised through a think-verify-revise loop. This addresses a key limitation of standard CoT prompting, which encourages step-by-step reasoning but lacks any built-in mechanism to check the logical validity of the steps it produces, making it more susceptible to compounding errors across a reasoning chain.

- **Persona patterns**, first discussed by White et al. [8], involve assigning the

model a specific role or identity within the prompt (e.g., "You are an experienced Python developer") to steer its output toward a particular style, level of expertise, or domain convention. The absence of an assigned persona is itself commonly cited as a prompt smell, as discussed further in Section 2.3.

- **Few-shot learning**, first analyzed by Brown et al. [9], refers to including a small number of example input-output pairs directly within the prompt, allowing the model to infer the desired pattern or format without additional training. This technique contrasts with zero-shot prompting, where no examples are provided and the model must rely solely on the instruction itself. Few-shot examples are particularly useful for enforcing a specific output format or coding convention that may not be reliably inferred from an instruction alone.

Together, these techniques illustrate that prompt quality is not solely a matter of clarity or grammatical correctness, but also depends on the structural and contextual choices made by the prompt's author. At the same time, poor prompt design can have the opposite effect and lead to inaccurate or incomplete results. This precisely explains why research into anti-patterns and "prompt smells" represents a new frontier in scientific research [10, 1].

2.3 Prompt Smells

Despite the widespread adoption of Large Language Models (LLMs) in software engineering workflows, the quality of the prompts used to query them has traditionally been assessed only indirectly, through the quality of the resulting output, rather than as a property of the prompt itself. Drawing an explicit analogy with the concept of *code smells* in software engineering, where a smell denotes a surface-level indicator of a deeper structural or design weakness without necessarily constituting an outright defect, Ronanki et al. [10] introduced the notion of *prompt smells* in the scientific literature as "*semantic and/or syntactic characteristics of a prompt instance resulting from (unintentionally) imprecise prompt engineering*." They are not actual errors — the prompt remains syntactically valid and will still produce an output — but "*can lead to issues related to (i) the desirability of the outputs, (ii) the lack of explainability*

of the generation process; or (iii) the traces between the input and the output, especially in Chain-of-Thought strategies."

In early 2026, Della Porta et al. [1] were among the first to treat the prompt as a software engineering artifact worth evaluating on its own, independently of the response it elicits. The authors conducted a rapid literature review to derive an initial catalog of eight prompt smells, each later operationalized through a dedicated metric. Prompt smells were grouped into two broad families: **syntactic smells** and **semantic smells**. The former concern the linguistic and structural well-formedness of a prompt, regardless of its content or intent:

- **Grammar and Surface-Form Smell**: misspellings, incorrect grammar, or malformed sentences that hinder clarity.
- **Formatting and Structural Smell**: inconsistent punctuation, capitalization, or layout that obscures the underlying task structure.
- **Readability Smell**: prompts that are overly complex, dense, or ambiguous, which can impair the model's ability to follow instructions correctly.
- **Overall Prompt Quality Smell**: not a smell in its own right, but a composite indicator obtained as the arithmetic mean of the three preceding syntactic smells, capturing baseline prompt quality.

Semantic smells, by contrast, concern higher-level, meaning-oriented properties that cannot be inferred from surface form alone:

- **Missing or Insufficient Context Smell**: the absence of information concerning the context of the request (e.g., missing objective, domain constraints, or data description).
- **Over-Complexity and Excessive Length Smell**: unnecessary verbosity or compound instructions that dilute the essential information in a request.
- **Style or Formality Mismatch Smell**: a mismatch between the style of the prompt and the style expected of the response.
- **Bias and Unsafe Framing Smell**: biased, unsafe, or discriminatory language that risks inducing harmful model behavior.

Table 2.1: Della Porta et al.'s catalog of prompt smells and metrics

Prompt Smell	Scale	Metric Computation (Scores)
Prompt Quality (PQS)	Continuous, $[0,1]$	$PQS = \frac{G+F+C}{3}$ *Where G: grammatical correctness; F: formatting quality; C: readability.*
Grammatical correctness (G)	Continuous, $[0,1]$	$G = 1 - \frac{n_{\text{matches}}}{\max(1, n_{\text{words}})}$ *Where* n_{matches}: grammar / spelling issues; n_{words}: *words.*
Formatting (F)	Continuous, $[0,1]$	*Normalised combination of punctuation, capitalisation and layout indicators.*
Readability (C)	Continuous, $[0,1]$	*Readability metrics*
Relevance Context (RC)	Continuous, $[0,1]$	$RCS = \sigma(s_{\text{llm}})$ $s_{\sigma(llm)}$: *LLM score for adequacy of explicit context.*
Complexity-Length (CL)	Continuous, $[0,1]$	$CLS = 1 - \min\left(1, \frac{\frac{WC}{WC_{\max}} + \frac{GFI}{20}}{2}\right)$ WC: *word count;* $WC_{\max}$: *length threshold;* GFI: *Gunning Fog Index.*
Formality Mismatch (FM)	Continuous, $[0,1]$	$FMS = \lvert s_{\text{formal}} - t_{\text{formal}} \rvert$ s_{formal}: *estimated prompt formality;* t_{formal}: *desired formality.*
Bias Detection (BD)	Continuous, $[0,1]$	$BDS = \alpha b_{\text{lex}} + (1-\alpha)\, b_{\text{llm}}, \quad \alpha \in [0,1]$ b_{lex}: *lexical bias indicator;* b_{llm}: *LLM bias score.*

Subsequently, each of these smells has been assigned an evaluation metric, as shown in Table 2.1. Interestingly, the empirical results reported in the research show that syntactic smells are strongly interrelated and largely predict a composite measure of prompt quality, while semantic smells behave independently of surface-level text properties and are not well predicted by them. This suggests that a prompt can look perfectly correct on the surface while still suffering from deeper, meaning-related issues that no amount of grammar or formatting analysis would catch.

2.4 Research Gaps

The study by Della Porta et al. is one of the first attempts to treat the prompt as an artifact whose intrinsic quality can be characterized and measured. Nevertheless, the authors are explicit about the scope of the work, describing it as a "preliminary exploratory study" and as an "early step" toward a principled understanding of

prompt quality. Indeed, we identified three gaps that the existing evidence leaves open and that motivate the work presented in this thesis.

- **Gap 1: The catalog of prompt smells is not systematic**, since it's derived from a rapid literature review rather than a systematic one. A rapid review is a research method that, by following less structured protocols than a systematic literature review, prioritizes speed over systematicity. The authors justify this choice by the fast pace of the field and by the impracticality of a full screening protocol, and they acknowledge that the resulting taxonomy is "intentionally preliminary" and not "a final or exhaustive catalog." As proof of this, their search on Google Scholar produced only 30 candidate articles, of which only 15 were selected for qualitative analysis.

- **Gap 2: The empirical evidence lacks large scale analysis.** The employed dataset consists of prompts written by developers and shared on public platforms and directed at a specific family of LLM-based tools. The authors themselves caution that the empirical findings may not generalize to other languages, organizational contexts - e.g., industrial settings and non-technical users - or model families. It remains open whether the findings are robust across larger, more heterogeneous corpora.

- **Gap 3: The effect and mitigability of prompt smells are unknown.** In software engineering, code smells are valuable because they are paired with refactorings that remove them while preserving behavior. For prompt smells, no equivalent has been proposed or evaluated. The study never applies any mitigation to a smelly prompt or measures the effect on the output. Consequently, we do not know whether the identified smells actually degrade output quality, whether they can be removed, manually or automatically, without altering the intent of the prompt, and whether their removal yields measurable improvements.

CHAPTER 3

Research Method

The purpose of the research presented in this chapter, i.e., the very purpose of this thesis, is to address the three research gaps identified in Section 2.4. First of all, three research question were defined, one for each research gap (Section 3.1). Subsequently, a tertiary study was conducted to find an answer to Research Question 1 (Section 3.2), leading to the compilation of a systematic, consolidated catalog of prompt smells. Building on this catalog, a detection algorithm was developed and applied to a large-scale dataset of real-world prompts to find an answer to Research Question 2 (Section 3.3). Finally, Research Question 3 was tackled by implementing an automatic correction mechanism for a subset of the cataloged smells, and by empirically validating its effectiveness on the same large-scale dataset through a second, independent detection pass (Section 3.4).

3.1 Research Questions

First of all, it was necessary to define a set of research questions that could address the current research gaps and guide our research. In total, the following three research questions were identified.

RQ$_1$. *What kind of prompt smells exists in the scientific literature and how can they be measured?*

As highlighted in Research Gap 1, the existing catalog of prompt smells by Della Porta et al. [1] was derived from a rapid literature review rather than a systematic one. To address this gap, it is necessary to conduct a systematic literature review to identify any articles that discuss bad practices that could compromise the quality of the output of Large Language Models, with a particular focus on articles dealing with software engineering. In addition, it is advisable to associate evaluation metrics with each prompt smell and consider a way to determine their values.

RQ$_2$. *What is the prevalence of prompt smells on a large scale dataset?*

As highlighted in Research Gap 2, the catalog mentioned above has not been evaluated on a large scale. To answer this question, we will need to identify a large-scale dataset of real-world prompts and implement an algorithm capable of detecting prompt smells within them. Thereafter, a statistical analysis shall be conducted to understand the distribution of prompt smells in the dataset and any correlations.

RQ$_3$. *Once smells have been identified in a prompt, to what extent can they be mitigated?*

To answer this question, it will be necessary to try to implement an algorithm capable of removing smells from a prompt automatically and determining whether they are actually detected following a second check.

3.2 Tertiary Review

In order to answer the first research question, it was necessary to conduct a systematic literature review to identify any articles that discuss bad practices that could have been useful to compile a catalog of prompt smells. To reduce biases related to subjectivity, the research was conducted in collaboration with a colleague that was also working on his thesis in this field. The scope of the review focuses specifically on articles that discuss prompt engineering and bad practices in software development workflows. Consequently, the approach we adopted for our research

follows the guidelines of Kitchenham and Charters [11] for conducting SLRs in the field of Software Engineering, and those of Wohlin [12] for applying the literature snowballing technique.

Given the sheer volume of articles addressing this topic, conducting a classical systematic literature review would have been excessively time-consuming. Fortunately, the scientific community has already conducted numerous SLRs within the field of prompt engineering. For this reason, it was decided to conduct a **tertiary review**, which, according to Kitchenham and Charters' definition, "*is a systematic review of systematic reviews, in order to answer wider research questions. A tertiary review uses exactly the same methodology as a standard systematic literature review.*"

The research process was structured into two main phases: (1) the *planning* phase and (2) the *conducting* phase and (3) *reporting* phase (see Section 4.1).

3.2.1 Planning Phase

In the initial phase of the research, we selected the scientific databases from which to retrieve articles and created a search string to query them, then we defined the inclusion and exclusion criteria.

Three databases were selected as sources for scientific articles: IEEE Xplore, Scopus, and ACM Digital Library. To query their search engines, we defined the following **search string**, with minor adjustments needed to adapt it to each search engine:

> *((prompt engineering OR prompt quality OR prompt pattern OR prompt smell OR prompt patterns OR prompt anti-pattern OR prompt learning OR prompt debt OR prompt management OR prompt best practices) AND (review of studies OR structured review OR systematic review OR literature review OR systematic literature review OR literature analysis OR in-depth survey OR literature survey OR meta-analysis OR analysis of research OR empirical body of knowledge OR overview of existing research OR body of published knowledge OR mapping OR mapping studies))*

The search process was conducted in February 2026. Using the search string, 224 articles were found on IEEE Xplore, 200 on Scopus, and 254 on the ACM Digital Library, resulting in a total of 678 articles. Among these, 18 articles were removed

because they were duplicates.

Subsequently, we defined the inclusion and exclusion criteria to decide which articles should be included and which should be excluded for the articles; these criteria are listed in Table 3.1.

Table 3.1: Inclusion and Exclusion Criteria.

Inclusion Criteria		Exclusion Criteria	
I1	The paper is a secondary study.	E1	The paper has less than six pages.
I2	The scope of the paper is prompt engineering, or prompt patterns, or prompt anti-patterns.	E2	The paper is superseded by a later version from the same authors.
I3	The scope of the paper is software development.	E3	The paper is a proceedings paper.
I4	The paper is written in English.		

3.2.2 Conducting Phase

During the conducting phase, we first selected the articles relevant to our research and excluded those that did not meet the inclusion and exclusion criteria. The selection process began with a preliminary screening in which only the titles, keywords, and abstracts of the articles were analyzed, followed by a full-text screening phase. Thereafter, the iterative "literature snowballing" strategy was applied to the articles that had passed the full-text screening phase. Finally, after completing the snowballing iterations, the information needed to compile the catalog of prompt smells was extracted for the next phase of the research.

Preliminary Screening

During the preliminary screening phase, my colleague (R2) and I (R1) analyzed the titles and abstracts of the articles we retrieved from scientific databases in blind mode. To begin with, a random sample of twenty-five articles was generated: the analysis of this sample revealed four cases of disagreement, which—given that nine other articles were conference proceedings—corresponds to a 25% rate of disagreement among the sixteen articles that were actually screened. To avoid contaminating subsequent samples, all articles containing the term "proceedings" in the title were first excluded;

after that, a second sample of twenty-five articles was created to identify and resolve any potential biases among the researchers before proceeding with the screening of all remaining articles. The analysis of this second sample revealed just two cases of disagreement, equivalent to a rate of 8%, a percentage low enough to proceed with the screening of all remaining articles. At this point, of the original 660 articles, 227 were excluded because they were conference proceedings, while another 41 had already been screened in the first two samples: as a result, only 392 articles remain to be screened. Analysis of this final sample revealed only twenty-two cases of disagreement, equivalent to a rate of 5.6%. Table 3.2 lists all instances of disagreement between the two researchers and how they were resolved. Column "S" indicates the sample to which the scientific article belongs. Columns "R1" and "R2" show the decisions made by the two researchers; the question mark ? in these two columns indicates that the researcher was unsure how to handle that article. Column "D" shows the final decision reached at the end of the discussion. In the end, out of a total of 433 articles (excluding conference proceedings), only 19 were accepted during this preliminary screening phase.

Full-text Screening

During the full-text screening phase, the full text of the articles that passed the preliminary phase was read and evaluated. Of the 19 initial articles, only 8 were selected as containing information relevant to our research. Several articles have been discarded because they discussed prompt engineering rather too broadly, without focusing on software development processes; others covered only prompt patterns or best practices, which was the opposite of what we were looking for. However, some of this information was considered useful, as it was deemed easy to derive an anti-pattern from it (e.g., the anti-pattern corresponding to "assigning a role to the LLM" is not assigning one).

Snowballing

Literature snowballing is an iterative approach used in SLRs to find other useful articles starting from a start set of articles selected in an earlier phase. For each

Table 3.2: Cases of disagreement during the preliminary screening phase

Article	S	R1	R2	D	Rationale
Leveraging LLMs for Systematic Reviews: Evaluating Venture Capital Investment Strategies Across Large Datasets	1	?	X	X	No I2
Prompt-imposed structure: a methodological evaluation of LLM-generated survey data for theory-based modeling	1	?	X	X	No I1
Enhancing healthcare resource allocation through large language models	1	?	X	X	No I3
A Survey on Large Language Models for Code Generation	1	✓	X	✓	Researcher 2 didn't know that surveys are a type of literature review
Fusing LLMs and diffusion models: A comprehensive survey of progress, challenges, and future directions in generative AI	2	✓	X	✓	Although it is not the central theme, the study does address prompt engineering
A Systematic Review of Prompt Injection Attacks on Large Language Models: Trends, Taxonomy, Evaluation, Defenses, and Opportunities	2	X	✓	✓	Even though the main topic isn't software development, certain prompt injection techniques could currently degrade the quality of any output
Designing intelligent chatbots with ChatGPT: a framework for development and implementation	3	?	X	✓	Implementing a chatbot can be considered a specific type of software development
The Interim Experiment Report: A Systematic Account of The Experimental Design of Large Language Model-Based Text-to-Model Approaches	3	?	X	X	No I3
A comprehensive taxonomy of prompt engineering techniques for large language models	3	?	✓	✓	Even though the main topic isn't software development, since it discusses prompt engineering in general, some sections may relate to that topic
Prompt Engineering or Fine-Tuning: An Empirical Assessment of LLMs for Code	3	X	?	X	No I1
Modern Approaches to Software Vulnerability Detection: A Survey of Machine Learning, Deep Learning, and Large Language Models	3	?	X	X	No I1
A Survey on Employing Large Language Models for Text-to-SQL Tasks	3	?	X	X	No I1
Large Language Models for Structured and Semi-Structured Data, Recommender Systems and Knowledge Base Engineering: A Survey of Recent Techniques and Architectures	3	?	X	X	No I1
Landscape and Taxonomy of Prompt Engineering Patterns in Software Engineering	3	?	✓	✓	The taxonomy was developed based on a SLR
A Meta-Analysis of Cloud-Based Llms in Implicit Multi-Step Reasoning	3	?	X	X	No I1
Diverse Prompts: Illuminating the Prompt Space of Large Language Models with MAP-Elites	3	X	?	X	No I1
Generative Artificial Intelligence for Software Development using ISO/IEC 29110 Basic Profile: Gaps and Opportunities	3	✓	X	✓	Although Researcher 2 had rejected it on the grounds of *No I2*, the study actually includes a SLR
A comprehensive survey of Vision–Language Models: Pretrained models, fine-tuning, prompt engineering, adapters, and benchmark datasets	3	X	✓	X	No I1 and No I3
Prompt Engineering in Education: A Systematic Review of Approaches and Educational Applications	3	X	✓	X	No I3
Understanding and Mitigating Hallucinations in Large Language Models: Insights from a Systematic Literature Review	3	✓	X	✓	Even though the main topic isn't software development, since it discusses LLMs hallucinations, the article could contain useful information
A Systematic Literature Review on Large Language Models Applications in Computer Programming Teaching Evaluation Process	3	✓	X	X	No I3
Developing an AI Readiness Model for Software Project Management: A Thematic Analysis	3	✓	X	✓	Even though the main topic is prompt engineering, the article could contain useful information
Prompt engineering in higher education: a systematic review to help inform curricula	3	X	✓	X	No I3
Prompting Literature Review: The Impact of Transformer Models and ChatGPT on Prompt Engineering and AI Research Publications	3	✓	X	✓	Even though the main topic isn't software development, the article may still contain sections that discuss it
Optimizing Generative AI Applications: A Comparative Study of Effective Prompting Techniques	3	X	✓	X	No I3
Large Language Models for Software Engineering: A Systematic Literature Review	3	✓	X	✓	Even though the main topic isn't prompt engineering, the article may still contain sections that discuss it
Navigating the Pitfalls: Analyzing the Behavior of LLMs as a Coding Assistant for Computer Science Students—A Systematic Review of the Literature	3	✓	X	✓	Even though limited to students, the topic is still related to software development
Screening Automation for Systematic Reviews: A 5-Tier Prompting Approach Meeting Cochrane's Sensitivity Requirement	3	X	✓	X	No I1

iteration, two phases are carried out: backward snowballing, in which older but relevant studies are sought by reviewing the references at the end of a previously found article; and forward snowballing, in which tools such as Scopus or Google Scholar are used to search for relevant articles among the ones that have cited that specific article. The iterative process ends when an iteration does not yield any useful articles. In our case, the initial set consisted of the eight articles that passed the full-text screening phase. Table 3.3 lists the nine articles taken into account during the backward snowballing phase of the first iteration, and the decision taken after the full-text analysis.

Table 3.3: Articles taken in account during the first backward snowballing iteration

Article	R1	R2	D	Rationale
A systematic survey of prompt engineering in large language models: Techniques and applications	✓	✓	✓	
If LLM is the wizard, then code is the wand: a survey on how code empowers large language models to serve as intelligent agents	✓	✓	✓	
The prompt report: A systematic survey of prompting techniques	X	✓	X	No I3
Unleashing the potential of prompt engineering in Large Language Models: a comprehensive review	X	✓	X	No I3
Siren's song in the AI ocean: a survey on hallucination in large language models	X	X	X	No I3
A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions	X	X	X	No I3
A comprehensive survey of hallucination mitigation techniques in large language models	X	X	X	No I3
ChatGPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design	X	✓	X	It discusses extremely specific prompt patterns (e.g., how to generate an API) and says nothing about bad practices
Large language models for software engineering: Survey and open problems	X	✓	X	It focuses more on LLMs than on how to prompt them

As for the forward snowballing, we selected an article from the start set and searched Google Scholar for articles that cited it using our initial search string; the search yielded a total of 1,030 articles. Consequently, given the large number of articles we would have had to analyze (more than double the initial number), we decided to abandon the forward snowballing phase. Surprisingly, the second iteration of backward snowballing did not produce any articles. In conclusion, the entire snowballing process allowed us to find two additional articles from which to extract information.

Data Extraction

Table 3.4 lists the articles that have passed the full-text screening phase. It can be observed that nine out of ten articles were published in 2025, while the other one was published in 2024.

Table 3.4: Final Articles from which data were extracted

ID	Article	Authors	Year	Reference
1	Understanding and Mitigating Hallucinations in Large Language Models: Insights from a Systematic Literature Review	Singh et al.	2025	[13]
2	Prompt Engineering for Requirements Engineering: A Literature Review and Roadmap	Huang et al.	2025	[14]
3	Prompts Engineering Challenges in Software Code Generation	Gutierrez Sarria et al.	2025	[15]
4	Prompt Engineering Guidelines for Using Large Language Models in Requirements Engineering	Ronanki et al.	2025	[16]
5	A comprehensive taxonomy of prompt engineering techniques for large language models	Liu et al.	2025	[17]
6	Landscape and Taxonomy of Prompt Engineering Patterns in Software Engineering	Sasaki et al.	2025	[18]
7	Prompting Techniques for Secure Code Generation: A Systematic Investigation	Tony et al.	2025	[19]
8	Unlocking the Potential of the Prompt Engineering Paradigm in Software Engineering A Systematic Literature Review	Syahputri et al.	2025	[20]
9	A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications	Sahoo et al.	2025	[21]
10	If LLM is the wizard, then code is the wand: a survey on how code empowers large language models to serve as intelligent agents	Yang et al.	2024	[22]

To facilitate the data extraction process, an extraction manual has been defined. In other words, every block of text deemed useful had to be classified into one of the following four categories: **Bad Practices**, **Best Practices**, **Challenges**, and **Ambiguities**. Once the relevant information from the articles were extracted, my colleague and I met to develop the catalog of prompt smells. The strategy we've adopted is the following: bad practices can be considered, basically, prompt smells themselves; best practices have been turned on their head - e.g., by reversing statements such as "Techniques such as CoT in particular, and other task-specific prompts, prove highly effective in enhancing factual reasoning." in [13], the prompt smell *Reasoning Suppression* was derived. As for ambiguity and challenges, we didn't identify many smells

from them, since they often consisted of very specific information, or information that contradicted that found in other articles.

After extracting the data, an initial list of eleven recurring bad practices was obtained from the selected articles. Only five of them were deemed measurable in any meaningful way and, interestingly, all five can be regarded as the counterpart of a corresponding prompt pattern: in other words, a given prompt smell is considered present whenever the associated prompt pattern is absent from the prompt. For this reason, the metric assigned to each of these five smells is a binary value that is `true` only if the prompt pattern is absent. The aforementioned list and the final catalog of prompt smells are shown in Chapter 4 — Analysis of the Results, respectively in Table 4.1 and Table 4.2. Furthermore, in the Section 4.1.1 of the same chapter, reports some interesting points of the ambiguities and challenges identified during the data extraction phase.

3.3 Large-scale Evaluation

Once the catalog of prompt smells had been compiled and each smell associated with its corresponding evaluation metric, the subsequent step consisted in measuring these metrics at scale, i.e., over a large volume of prompts. To this end, it was first necessary to select a sizeable dataset of real-world prompts; subsequently, an algorithm were implemented to detect the presence of prompt smells within them.

3.3.1 Dataset Selection

The dataset employed to analyze the prevalence of prompt smells in requests related to the field of software development is a subset of *CodeChat-V2.0* derived by Francesco Pinto while working on his Master's thesis [23].

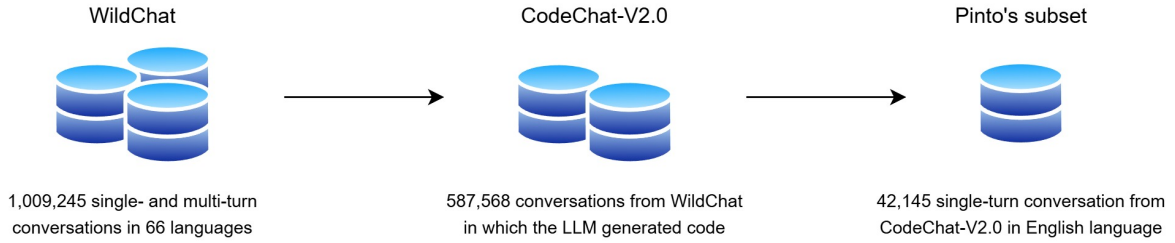
As stated by its creator, *CodeChat-V2.0* is a large-scale dataset comprising 587,568 real-world developer–LLM conversations, derived from the *WildChat* dataset. In turn, *WildChat*'s creators collected 1,009,245 conversations between human users and ChatGPT by offering online users free access to OpenAI's GPT-3.5 and GPT-4 [24].

To identify code-related conversations in *WildChat*, the creators of CodeChat

examined the content field of each LLM turn: if at least one LLM turn contained a block of code enclosed in three backticks (i.e., ```) followed by a programming language identifier (e.g., Python), the conversation was considered code-related [25].

In his work, Pinto used the *CodeChat-V2.0* dataset as a starting point and, through data selection and preparation, was able to produce a final dataset consisting of 42,145 lines of single-turn developer-LLM conversations written in English.

Figure 3.1: Procedure for obtaining the final dataset



To achieve this, he started with an initial filtering process, necessary to reduce the size of the original dataset; two types of filters were applied:

- **Turn filter**: he decided to consider only single-turn conversations, so as not to influence the result based on previous prompts. Thanks to this filter, approximately 57,349 multi-turn conversations were removed from the dataset.
- **Language filter**: since some metrics are defined only for English texts, he decided to consider only English-language conversations. Thanks to this filter, approximately 332,346 conversations were removed.

At the end of this first filtering stage, the dataset was reduced to approximately 197,873 conversations.

The second step consisted in the de-duplication of the dataset: 26,974 conversations that are identical to one another were removed, along with another 109,064 that, while not perfectly identical, contained minor textual differences, identified using *MinHash* and *Locality-Sensitive Hashing* (LSH) algorithms. The goal was to reduce biases during the analysis phase, caused by the over-representation of specific cases. Following these two de-duplication stages, the dataset was reduced to 61,835 conversations.

In the next step, Pinto applied operations designed to split users' requests written in natural language from code snippets within the remaining prompts. The main reason behind this decision was to ensure an accurate calculation of the metrics, without taking into account code snippets and, above all, code comments. To carry out this operation, he employed an approach adopted by Della Porta et al. [26], which involves using an LLM-based procedure to classify each portion of a prompt as either natural language or code. At the end of this complex procedure, a sample of conversations was analyzed to verify that the classification was correct.

In this stage, during the analysis of a sample from the filtered dataset, he noted an anomaly: there were still conversations in languages different from English. This problem was likely caused by the presence of code inside the prompt. After resolving this anomaly, he performed additional operations to label the prompts based on the task requested and the topic. As mentioned at the beginning, the initial dataset was reduced to **42,145** single-turn developer–LLM conversations, written in English.

3.3.2 Which Prompt Smells Can Be Detected, And How?

As described afterwards in Section 4.1, only five of the prompt smells listed in Table 4.1 were associated with an evaluation metric, namely *Reasoning Suppression*, *Lack of Self-Reflection*, *Role Suppression*, *Unspecified Output Structure*, and *Lack of Examples*. The metric adopted for these smells is binary, as it merely establishes whether the corresponding smell is present within the prompt. More precisely, the metric evaluates to `true` when the associated prompt pattern is absent; for instance, in the case of *Role Suppression*, the metric holds `true` if no role or persona has been assigned to the queried LLM. Identifying these prompt patterns is expected to be relatively straightforward, as each of them is typically conveyed through recognizable phrasings: role assignment, for example, is commonly expressed through phrases such as "Act as..." or "You are an...", whereas reasoning elicitation is typically conveyed through phrases such as "Let's think step by step" or "Explain your reasoning". For this reason, it was decided to try two different approaches for detecting these patterns: one based on regular expressions (see Section 3.3.3), and one based on an LLM-as-a-judge approach (see Section 3.3.4).

Alongside the aforementioned five smells, we also wondered whether some of the smells from the catalog by Della Porta et al. could have been included. Their catalog lists eight prompt smells: *Grammar and Surface-Form*, *Formatting and Structural*, *Readability*, *Over-Complexity and Excessive Length*, *Missing or Insufficient Context*, *Style or Formality Mismatch* and *Bias and Unsafe Framing*. Each of these smells is associated with a metric that can be measured continuously on the interval $[0, 1]$; however, as pointed out by the researchers, the last three smells — classified as "semantic" — require LLM-based techniques to be detected. Given the non-deterministic nature of LLMs — especially regarding topics that are open to interpretation even for humans, such as bias detection — it was decided to not take in account these last three in favor of the first five, whose metrics can be calculated deterministically.

The *Formatting* score, as proposed by Della Porta et al., could be computed as a normalized combination of punctuation, capitalization and layout indicators within a prompt. We believe that punctuation and capitalization should be given greater weight than layout indicators: specifically, a weight of 40% for the first two and 20% for the latter.

Again, as proposed by the authors, the *Gunning fog index* could be employed to compute the *Complexity-Length* score. This index, developed by American businessman and textbook publisher Robert Gunning in his 1952 book "*The Technique of Clear Writing*", serves to estimate the years of formal education required for a reader to comprehend a given English text on the first reading. [27]

As for the *Readability* score, who measures how easy is to read the prompt, it could be computed using the *Flesch Reading Ease* index, a value on a scale ranging from 0 to 100 first introduced by the Austrian linguist Rudolf Flesch [28].

Finally, to calculate the score of the *Grammatical Correctness* score - which quantifies the density of grammar and spelling issues - the famous tool *LanguageTool* could be employed. It is an open-source grammar, style, and spell checker software developed in 2003 by Daniel Naber for his master thesis and improved over the years. [29, 30]

For each of the five syntactic metrics, a threshold was established such that, if not exceeded, it signals the presence of the prompt smell associated with that metric. The *Grammatical Correctness Threshold* was set to 0.9 so as to tolerate only minor, physiological typos; falling below this threshold means having a grammatical error

density exceeding 10% of the text. The *Formatting Threshold* was set to 0.75 to ensure that at least three-quarters of the prompt adhere to basic typographical conventions. The *Readability Threshold* was set to 0.6 because, since the metric is derived from the normalized *Flesch Reading Ease* score, texts scoring below 60 are classified as difficult to read and rich in complex sentence structures. The *Prompt Quality Threshold* equals 0.7, i.e., the arithmetic mean of the three preceding thresholds. Finally, the *Complexity Length Threshold* was set to 0.75 since the metric combines Word Count and the Gunning Fog Index, and a value this close to 1.0 indicates a text that is either unnecessarily verbose or syntactically overly complex.

3.3.3 Regular Expressions Baseline

The first, fairly simple approach we took to detect the five prompt smells identified during our tertiary review was to use regular expressions. A regular expression (often shortened as *regex*) is a sequence of characters that specifies a set of strings that matches it. Python provides a library called *re* that contains several useful functions for checking whether a particular string matches a given regular expression.

For each of the five prompt smells, an array of regular expressions was defined based on our knowledge of the specific prompt pattern and the most common phrases found in the literature. For instance, the following code snippet shows the array of regular expressions defined for detecting the *Role Suppression* smell.

```
role_patterns = [
    r"\b(?:act|working)\s+as\s+(?:a|an)\b",
    r"\byou\s+(?:[''']ll|will)\s+act\s+as\b",
    r"\b(?:act)?as\s+(?:a|an)?\s+expert\s+in\b",
    r"\b(imagine\s+)?you\s+(?:[''']re|are)\s+(?:a|an)\b",
    r"\b\bpretend\s+(?:to\s+be|you(?:\s+are|[''']re))\b",
    r"\b(?:assume|take\s+on)\s+the\s+role\s+of\b",
    r"\bbehave\s+like\s+(?:a|an)?\b",
    r"\b(?:persona|role)\s*:\s*\w+",
    r"\bin\s+your\s+capacity\s+as\b",
    r"\bin\s+the\s+role\s+of\b"
]
```

Although implementing and testing an algorithm that uses regular expressions is relatively quick, these expressions present two significant structural limitations.

- False negatives: The main drawback of regular expressions is their inherent rigidity, which prevents them from covering all possible stylistic variations and edge cases related to the five prompt smells. For example, in order to capture every possible formulation of role attribution or self-reflection, an increasingly complex set of rules is required—one that, no matter how hard we try, inevitably remains incomplete. Moreover, common grammatical abbreviations, informal shorthand, or colloquial typos (for example, “imagine u are” instead of “imagine you are”) could lead to false negatives—that is, cases in which a pattern is present but is not detected by the pattern recognition system.

- False positives: albeit in theory this scenario was much less likely than the other one, there is a risk of detecting text patterns that did not match the prompt pattern. For example, one of the regular expressions we associated with the *Lack of Examples* smell was "*e.g.*". If the user had used this abbreviation to better describe something regarding the context of the request, it would have been incorrectly detected as an example. Consider the following input: "*This text analysis software can be used in many ways,* ***e.g.****, to review students' essays or to perform sentiment analysis on product reviews.*" This would have been detected as a false positive.

Despite these risks, an experiment with regular expressions was nevertheless conducted in order to establish a baseline. The dataset used is the subset of *CodeChat-V2.0* created by Francesco A. Pinto (see Section 3.3.1) which, as we recall, contains 42,145 single-turn conversations between developers and LLMs. The prompt smells with the highest percentages of occurrence rates are *Lack of Self-Reflection* (detected in 99.00% of the prompts) and *Reasoning Suppression* (98.60%), followed by *Unspecified Output Structure* (93.70%), *Lack of Examples* (89.20%) and *Role Suppression* (87.40%).

3.3.4 LLM-as-a-Judge Approach

In light of the risks outlined in the previous section, it became clear that the regular expression-based approach had significant limitations. For this reason, we decided to shift towards the LLM-as-a-judge paradigm. Nevertheless, it is important to consider that, since this is an LLM-based approach, the *Naso* tool's output inevitably becomes

non-deterministic, i.e., by feeding the same prompt to the tool multiple times, it might return diagnoses that differ from one another.

Statistical Analysis and Model Selection

Before selecting the model, it was necessary to conduct a quick statistical summary of length of the prompts in the dataset. The mean value is approximately 6,643; the median (i.e., the 50th percentile) and the 80th percentile, whose values are respectively 746 and 6,337, are both smaller than the mean. From this, it can be assumed that prompts that raise the mean are less than 20% of the entire dataset. Finally, the 95th and 99th percentiles are respectively 29,717 and 90,318, while the maximum value is 784,259. Since the *CodeChat-V2.0* dataset consists of real-world interactions between developers and LLMs, it can be estimated that 95% of the prompts written by users will be under 16,000 characters long. Considering that for English texts a token corresponds to approximately four characters, and that furthermore the LLM judge doesn't need to keep track of any context, a small model like Qwen2.5 is considered a good trade-off between reliability and speed [31]. In particular, the 7B-version of this model can support up to 128K tokens. Qwen2.5 is an open-weight large language model developed by the Qwen Team at Alibaba Group and released in 2024. As stated by the developers, this version demonstrates significantly more knowledge compared to Qwen2 and possesses markedly enhanced capabilities in programming and mathematics. Furthermore, there are significant improvements in instruction following, generating long texts (over 8K tokens), understanding structured data and generating structured outputs, especially JSON.

Defining prompt instructions for the LLM Judge

The first step was to craft the instructions to query the LLM judge to classify the five dimensions. Since we know very well that the quality of the input significantly affects the quality of an LLM's output, to ensure that the outputs were as accurate as possible, this prompt required meticulous engineering. This was achieved by combining my knowledge in prompt engineering with a technique known as *meta-prompting*. Meta-prompting is a prompt engineering methodology wherein an LLM is employed

to generate, refine, or optimize instructions for itself or, as in this case, another model. In this research, a baseline prompt was initially drafted and subsequently refined through iterative interactions with an LLM, leveraging its analytical capabilities to enhance the clarity, structure, and constraints of the instructions. Afterwards, two examples were manually added to the prompt to prevent the meta prompter from introducing hallucinations.

Refining prompt instructions

Initially, again using meta-prompting, an attempt was also made to include into the prompt a block requesting that the confidence score should be returned. However, after conducting a couple of experiments on small samples, it was noted that almost all the responses had a confidence level of 90%, with the exception of a few that had a confidence level of zero. In light of this, we chose to exclude the confidence calculation from the process, thereby reducing the number of tokens fed to the LLM judge as input.

Three experiments were conducted to evaluate the LLM-as-a-judge approach and, at the same time, refine the crafted prompt. At the end of each experiment, the same sample of 100 randomly selected prompts was examined to determine the extent to which and in what ways the LLM model tended to hallucinate.

Following the first experiment, we observed hallucination mainly with regard to the *Reasoning Suppression*, *Role Suppression*, and *Unspecified Output Structure* prompt smells. For this reason, we refined the instructions by improving the definitions of these prompt smells.

Following the second experiment, we realized that the LLM tended to hallucinate, especially when detecting *Reasoning Suppression* because it confused its own reasoning process with the one expected—but not actually present—in the user's prompt. For this reason, the constraint "*When assigning a value to the 'is_reasoning_requested' field, consider only any user requests made to the LLM—not your own reasoning regarding the 'reasoning' field.*" was added to the instructions.

Finally, following the third experiment, the LLM appeared to generate substantially fewer hallucinations compared to the other two experiments. However, it was

necessary to empirically verify whether the LLM judge's classifications were indeed reliable. To this end, I enlisted the help of the colleague who collaborated with me on the tertiary review (see Section 3.2). We independently classified the sample of 100 prompts mentioned above across the five dimensions and then computed the Cohen's kappa coefficient between our own evaluations. Cohen's kappa (κ) is a statistical measure used to assess the level of agreement between two raters classifying items into categorical classes, correcting for the agreement that could occur by chance alone [32]. A value of $\kappa = 1$ indicates perfect agreement, while $\kappa = 0$ indicates agreement equivalent to chance. According to the widely adopted interpretation scale proposed by Landis and Koch [33], values between 0.61 and 0.80 indicate substantial (good) agreement, whereas values above 0.80 indicate almost perfect agreement.

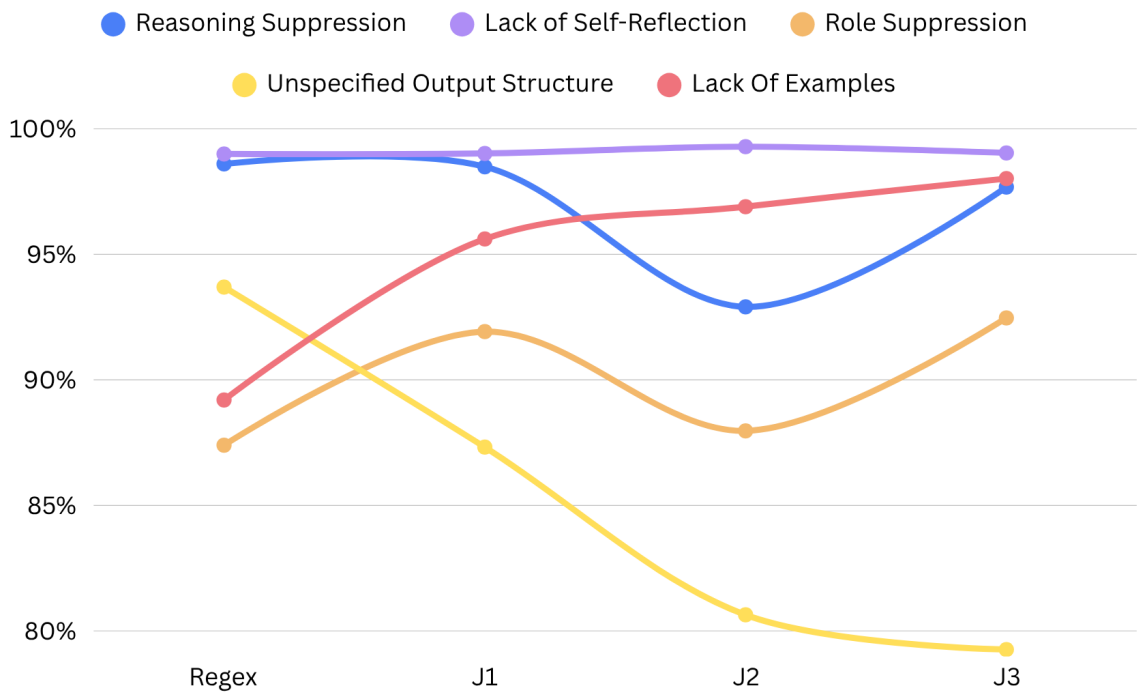
Based on our classifications, the Cohen's kappa that my colleague and I obtained was $\kappa = 0.77$ — indicating good agreement, and thus sufficient to proceed further. We then computed Cohen's kappa coefficient between each of our evaluations and those of the LLM judge, obtaining $\kappa = 0.68$ for mine and $\kappa = 0.64$ for my colleague's. These values are also considered indicative of good agreement.

Table 3.5 shows the percentage of prompt smells detected in the dataset after each experiment, whereas Figure 3.2 shows the a line plot based on this data. Column *R* reports the percentages of smells detected through the regex-based approach, while columns *J1*, *J2*, and *J3* report the results of the three LLM-as-a-judge experiments.

Table 3.5: Percentages of prompt smells detected in multiple experiments

Prompt Smells	R	J1	J2	J3
Reasoning Suppression	98.60%	98.49%	92.91%	97.68%
Lack of Self-Reflection	99.00%	99.02%	99.29%	99.04%
Role Suppression	87.40%	91.92%	87.97%	92.47%
Unspecified Output Structure	93.70%	87.32%	80.64%	79.26%
Lack of Examples	89.20%	95.61%	96.90%	98.02%

Figure 3.2: Line plot of percentages of prompt smells detected in the experiments



It can be observed that the percentages for the *Lack of Self-Reflection* smell remained consistently above 99% across all experiments. Within the aforementioned sample of 100 prompts, no disagreement with the LLM judge was found following any of the three experiments. This indicates that the user requests the LLM to double-check its own output in only 1% of prompts.

With regard to the *Lack of Examples* smell, the percentages increased from one experiment to the next, reaching 98% in the final experiment. This increase can be attributed to the fact that the LLM judge frequently confused the user's code copy and pasted in the prompt with examples. Following the final experiment, only five samples were found to be in disagreement between my evaluation and that of the LLM judge.

Conversely, the percentage for *Unspecified Output Structure* decreased across successive experiments. This is believed to stem from the LLM judge's initial inability to detect cases such as "*Write a code in [some language].*" This smell also exhibited the highest disagreement rate between my evaluation and the LLM judge's, with 16 samples out of 100 disagreeing.

Finally, both *Role Suppression* and *Reasoning Suppression* improved considerably in

the final experiment: for both smells, only 2 samples out of 100 resulted in disagreement.

Figure 3.3 shows the final version of the instructions to which the user's prompt must be appended before being provided to the LLM judge.

3.4 Smells Mitigation

The third gap identified in Chapter 2.4 concerns the mitigability of prompt smells: while Della Porta et al. [1] lay the conceptual groundwork for detecting such smells, they leave entirely open whether, and how, they can actually be removed from a prompt. RQ3 addresses this gap directly, asking whether, once a smell has been identified, it is possible to mitigate it, and requiring the implementation of an algorithm capable of automatically removing smells from a prompt and of verifying, through a subsequent check, that they are no longer detected. Out of the ten prompt smells composing the final catalog, it was possible to fix three of them, namely *Role Suppression*, *Reasoning Suppression*, and *Lack of Self-Reflection*. The remainder of this section first describes the correction mechanism adopted for these three smells (Section 3.4.1), and then discusses the reasons that led us to exclude the remaining seven smells from the correction pipeline (Section 3.4.2).

3.4.1 How are smells corrected?

The correction is performed by inserting a short sentence into the prompt for each of the detected smells. Specifically, for the *Role Suppression* smell, the sentence "`Act as an experienced software engineer.`" is appended at the beginning of the prompt; as for the *Reasoning Suppression* smell, the sentence "`Let's think step by step.`" is appended to the end of the prompt, whereas for the *Lack of Self-Reflection* smell, the sentence "`Review your output before replying.`" is appended to the end of the prompt, immediately after the reasoning-related addition. Since the research focuses on fixing prompt smells within prompts intended for software development and code generation, adopting a generic role such as that of a software engineer is believed to represent the best compromise for covering all

Figure 3.3: The final version of the instructions to be given to the LLM judge

You are an expert data annotator in natural language processing with an in-depth knowledge of prompt engineering. You will be given a prompt for a Large Language Model (LLM): your job is to analyze the prompt and extract 5 specific dimensions based on the prompt patterns described below.

DEFINITIONS & PATTERNS TO DETECT:

1. is_role_assigned: True if the prompt uses a Persona/Role Pattern, explicitly instructing the LLM to adopt a specific identity, expert role, or persona (e.g., "Act as...", "You are an expert...", "Pretend to be...").
2. is_reasoning_required: True if the prompt explicitly uses a Chain-of-Thought (CoT) pattern, instructing the LLM to break down its reasoning step-by-step before providing a final answer (e.g., "Let's think step by step", "First... Then...", "Explain your reasoning step by step").
3. self_reflection_present: True if the prompt uses a Self-Correction / Self-Reflection pattern, instructing the LLM to verify, review, critique, or double-check its OWN output before finalizing the response (e.g., "check to see if your answer is correct", "review your code before replying"). False if the LLM is only asked to review code provided in the user's input.
4. structure_specified: True if the prompt uses an Output Format Constraint pattern, explicitly requesting a specific structural format or constraint for the response (e.g., "output format: JSON", "structure the output as a table", "respond only in Python code").
5. examples_count: Exact integer count of input-output or few-shot examples provided within the prompt (e.g., "Input: ... Output: ...", "Example 1: ..."). 0 if none.

IMPORTANT:

– Write the 'reasoning' field FIRST in your JSON output to analyze the prompt step-by-step before assigning boolean and integer values.
– Some prompts may contain code snippets. You must not take any code strings or comments into account for your analysis under any circumstances!
– When explaining your reasoning and, be careful not to confuse the user's request with your job as an annotator.
– When assigning a value to the 'is_reasoning_requested' field, consider only any user requests made to the LLM—not your own reasoning regarding the 'reasoning' field.

EXAMPLES:

1. Prompt: Act as an expert Python programmer. Write a Python script to calculate the n-th Fibonacci number. Examples: fib(1)=1, fib(4)=3
   Response: { "reasoning": "The prompt explicitly assigns the persona of an expert Python programmer ('Act as an expert...'). No Chain-of-Thought or self-reflection is requested. It asks for code, specifying the expected output structure. Two examples are explicitly provided.", "is_role_assigned": true, "is_reasoning_required": false, "self_reflection_present": false, "structure_specified": true, "examples_count": 2 }
2. Prompt: I want u to write code to calculate the factorial of a number. For example, if the user input is 5 the output must be 120. Explain the reasoning behind the code and double-check that it's correct.
   Response: { "reasoning": "No persona/role is assigned to the model. The prompt asks to explain the reasoning behind the code (Chain-of-Thought requested) and explicitly instructs the model to double-check its own output (Self-reflection pattern). No specific output structure is constrained. One example of input/output is provided.", "is_role_assigned": false, "is_reasoning_required": true, "self_reflection_present": true, "structure_specified": false, "examples_count": 1 }

possible cases.

Naturally, if only one or two out of the three smells are detected in a given prompt, the tool only appends the sentences corresponding to those smells, without overwriting patterns already present in the original prompt.

3.4.2 Why aren't the other seven smells corrected?

Unspecified Output Structure and Lack of Examples

The correction of the other two prompt smells, i.e., *Unspecified Output Structure* and *Lack of Examples*, was not as straightforward as that of the previous three. The structure of the output depends strictly on the user's needs: it should be the user who specifies, for instance, which programming language the requested code is expected to be returned in, or whether a different format is preferred (e.g., JSON or tables).

As for the *Lack of Examples* smell, it would have been possible to employ an additional LLM-as-a-judge to generate examples based on the user's request. However, as also highlighted afterwards in Section 4.1.1, the human user should already exercise extreme care when including examples in prompts, since incorrect or biased examples could degrade the quality of the LLM's output rather than improve it. Relying on an LLM to generate synthetic examples from a relatively short piece of text such as a prompt could therefore be a risky approach.

For the reasons mentioned above, we decided not to include these two prompt smells in the correction pipeline.

Syntactic Smells

As for the five syntactic smells, an attempt was made to address the *Grammar and Surface-Form* and *Readability* smells by using the `LanguageTool.correct()` function. Improving the score of the metrics associated with these smells could also have increased the Prompt Quality Score, another metric associated with the syntactic smell Prompt Quality, since this value is obtained by calculating the arithmetic mean of the Grammatical Correctness, Formatting, and Readability scores.

However, when this feature was tested on a simple prompt prior to being applied to the entire dataset, it became apparent that this approach could introduce bias.

Indeed, the sentence "Letis think step bay stp" was corrected to "Lewis think step bay STP" rather than to "Let's think step by step." In light of this, it could be hypothesized that LanguageTool's grammar checker might also erroneously modify portions of code detected as errors (e.g., variable or function names).

Finally, with regard to the *Formatting and Structural* and *Over-Complexity and Excessive Length* smells, an LLM-based approach could similarly have been employed to make the prompt more readable and concise. Nevertheless, as previously discussed, this non-deterministic approach carries the risk of introducing bias if not properly controlled.

CHAPTER 4

Analysis of the Results

4.1 RQ1: A Systematic Catalog Of Prompt Smells

To address this question, after completing the data extraction phase and creating the unprocessed catalog shown in Table 4.1, we considered which of the eleven smells were actually measurable.

The sixth, eighth and ninth were discarded as they were considered too general: for example, none of the articles explained the boundary between a prompt with enough context and one without. As opposed to those mentioned above, the tenth and the eleventh smells were considered too specific: as for the former, it is very likely that most developers are unaware of the known bugs related to the code they are asking to be generated, or that they may mention unrelated bugs; as for the latter, a certain level of expertise is required to structure a prompt properly according to SCoT rules (see Section 4.1.1). Potentially, a prompter who fails when trying to follow these two rules might end up introducing bias into their prompt. Finally, the seventh smell, which concerns the configuration of output hyperparameters (e.g., temperature), was excluded because not all LLMs —especially cloud-based ones like ChatGPT and Gemini — allow to configure these parameters.

Table 4.1: A first, unpolished catalog of bad practices in prompt writing

ID	Prompt Smells	Articles
1	Do not break complex reasoning tasks into stepwise reasoning paths.	1, 2, 4, 5, 7, 8
2	Do not prompt the LLM to engage in self-reflection (or self-consistency, or self-debugging) on the output it will generate.	1, 2, 4, 6, 9, 10
3	Do not assign a role to the LLM.	2, 4, 5, 7, 10
4	Do not specify the format of the output structure.	2, 4, 5
5	Do not provide examples.	7
6	Provide an underspecified prompt with lack of clarity and/or no additional description.	1, 3, 4, 5, 9
7	Hyperparameters like temperature that are unadjusted or poorly adjusted.	1, 8
8	Do not provide enough context.	2, 4, 5, 6
9	Provide a too much-detailed prompt.	3, 9
10	Do not include comments about known bugs and how to handle them in code-generation tasks.	6
11	Use the classic CoT instead of the Structured CoT.	9

👉 **Answer to RQ$_1$.** Eleven prompt smells were identified at the conclusion of the SLR, even though only five of them were actually considered to be associated with an evaluation metric: (1) Reasoning Suppression, (2) Lack of Self-Reflection, (3) Role Suppression, (4) Unspecified Output Structure, (5) Lack of Examples.

Table 4.2: The final systematic prompt smells catalog

Name	Smell Description	Evaluation Metric
Reasoning Suppression	Do not break complex reasoning tasks into stepwise reasoning paths.	Binary value: 0 if the LLM is not asked to reason (CoT or its variants), 1 otherwise
Lack of Self-Reflection	Do not prompt the LLM to engage in self-reflection on the output it will generate.	Binary value: 0 if not prompted to engage in self-reflection, 1 otherwise
Role Suppression	Do not assign a role to the LLM.	Binary value: 0 if no role/persona is assigned to the LLM, 1 otherwise
Unspecified Output Structure	Do not specify the format of the output structure.	Binary value: 0 if no output structure is specified, 1 otherwise
Lack of Examples	Do not provide any example.	An integer representing the count of examples

4.1.1 Ambiguity and Challenges identified

During the data extraction phase, we have identified some information that either raises ambiguities regarding most of the statements collected or presents open challenges for researchers in the field of prompt engineering for software engineering.

Liu et al. and Syahputri et al. both pointed out how Chain of Thought prompting, despite its efficacy in natural language generation tasks, demonstrates lower accuracy and security issues when applied to code generation tasks [17, 20]. Consequently, numerous prompting techniques have been designed to improve the classic Chain of Thought, such as the *Logical Chain of Thought* and the *Structured Chain of Thought* techniques (see Section 2.2). However, writing prompts that accurately reflect these advanced techniques takes longer than usual [21].

Nevertheless, a significant number of articles highlight the importance of this prompting technique, which is why its absence from a prompt was still included in the catalog as a smell.

💬 **Take-away Message #1.** The traditional Chain of Thought may not be as efficient at code generation tasks as it is at natural language generation tasks.

Ronanki et al. [16] and Liu et al. [17] both pointed out an issue regarding the few-shot prompting technique. While it can improve output quality by helping models comprehend task intent and standardizing output formats for complex tasks, its effectiveness remains heavily dependent on specific conditions and introduces severe token overhead that limits long-text processing. Consequently, *Instruction Understanding* has emerged as a promising alternative paradigm to overcome these demonstration-based limitations without incurring high token costs [17, 34].

💬 **Take-away Message #2.** Care must be taken with both the quality and quantity of examples included in the prompt.

In the same research, Ronanki et al. also highlighted the need for either developing new prompt engineering guidelines, specific to requirements engineering or fine-tuning existing guidelines to make them more applicable and effective for requirements engineering [16].

Tony et al., who focused on the topic of LLMs for secure code generation tasks, believe that including examples in prompts would be counterproductive because it would require either a deep understanding of software security on the part of the person writing the prompt, or readily available secure code examples—which is often unrealistic. During their experiment, a rather curious ambiguity also came to light: even though they employed the persona of a "software security expert" to prompt LLMs towards generating security-conscious code, this approach consistently performed the worst in terms of weakness count, rate, and density by all the LLMs [19].

4.2 RQ2: On Large-Scale Assessment of Prompt Smells

Answering this question required identifying a sizeable dataset of real-world prompts, developing and applying a detection algorithm to it, and subsequently performing a statistical analysis to characterize the distribution of prompt smells across the dataset, as well as any correlations among them. The remainder of this section first describes the application of the detection algorithm to the selected dataset and reports the resulting prevalence rates (Section 4.2.1), then analyzes the correlations observed among the different smells (Section 4.2.2), and finally summarizes the findings with respect to RQ2.

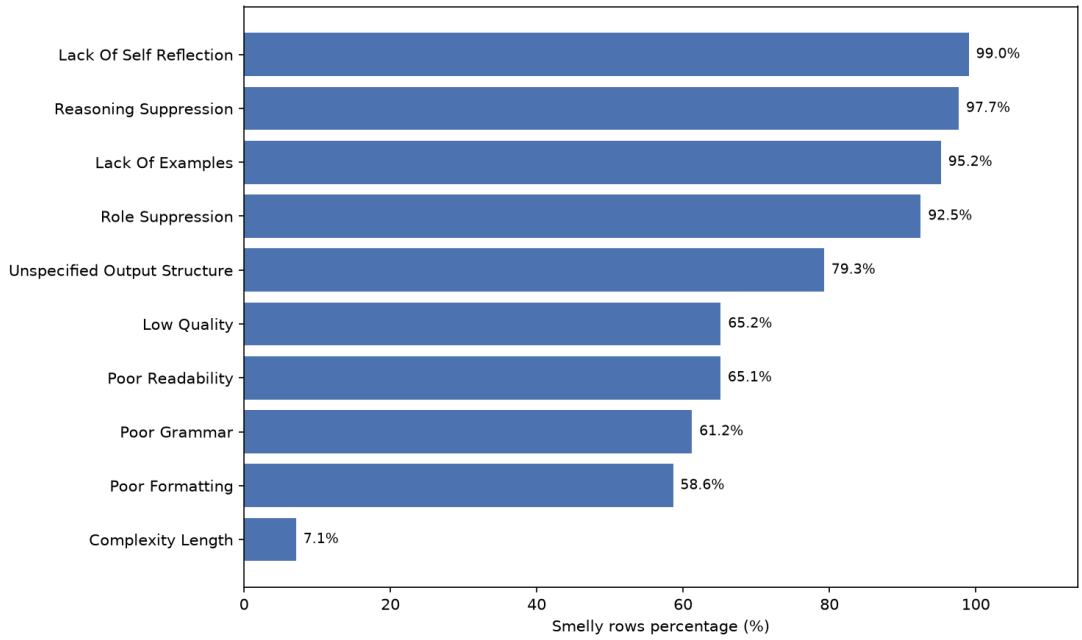
4.2.1 Application to a Large-Scale Dataset

To address this question, we employed the detection algorithm described in Section 3.3, specifically its LLM-as-a-judge-based variant, to obtain the values of all metrics and the corresponding prompt smells for the 42,145 prompts composing the *CodeChat-V2.0* subset. Table 4.3 reports the number of prompts in which each smell was detected, while Figure 4.1 shows the corresponding detection rates across the entire dataset. It should be noted that five prompts were excluded from the count due to the presence of at least one null value, bringing the total number of analyzed prompts to 42,140.

Table 4.3: Count of prompt smells detected in the dataset

Anti-Pattern Smells	Count	Syntactic Smells	Count
Reasoning Suppression	41163	Complexity-Length	2996
Lack of Self Reflection	41736	Poor Grammar	25799
Role Suppression	38969	Poor Readability	27439
Unspecified Output Structure	33402	Poor Formatting	24708
Lack of Examples	40134	Low Quality	27435

Figure 4.1: Detection rate for each prompt smell



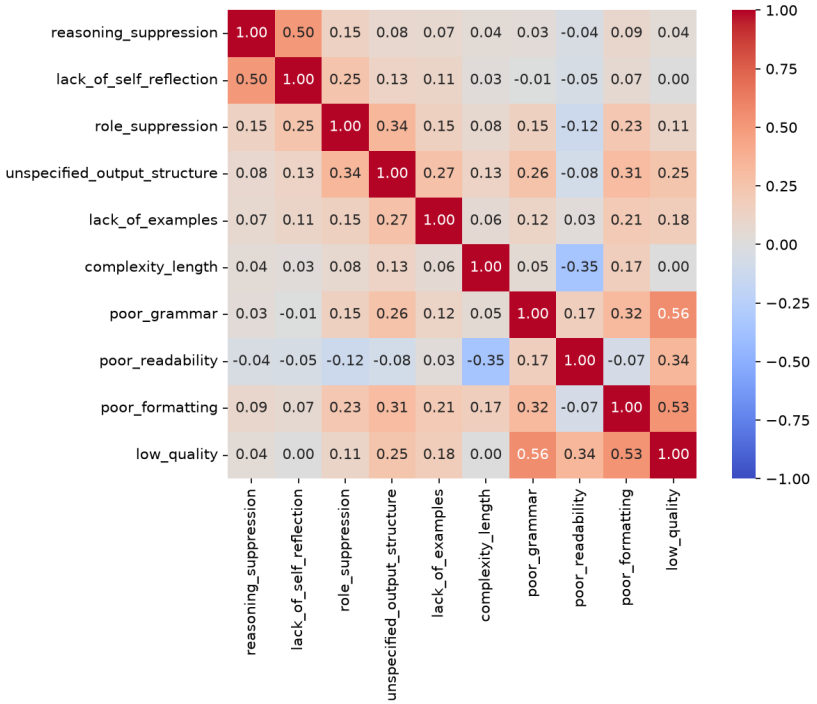
The five smells related to the absence of a specific prompt pattern exhibit the highest prevalence rates, whereas the four syntactic smells range between 58.6% and 65.2%. Surprisingly, however, the *Complexity-Length* smell exceeds the predefined threshold of 0.75 in only 7.1% of cases. This unexpectedly low prevalence prompted a closer examination of the underlying metric's distribution. The corresponding score exceeds the threshold value only from the 93rd percentile onward, while its 75th percentile corresponds to 0.54 and its median is equal to 0.0, indicating that the metric

is null for at least half of the dataset. The mean value of the *Complexity-Length Score*, in turn, is slightly below 0.25. Given that the *Readability* smell, by contrast, is detected in 65.1% of prompts, these results suggest that either the threshold or the formula underlying the *Complexity-Length* metric warrants reconsideration.

4.2.2 Correlation Between Prompt Smells

Figure 4.2 shows the correlation matrix constructed from the data collected in the previous phase.

Figure 4.2: Correlation matrix between prompt smells



The correlation matrix reveals that the majority of prompt smells exhibit weak to negligible pairwise correlations, mostly below 0.20 in absolute value, suggesting that the identified smells largely capture distinct and non-redundant dimensions of prompt design.

The strongest relationship observed is between *Reasoning Suppression* and *Lack of Self-Reflection* ($r = 0.50$), indicating that prompts which restrict the model's reasoning process also tend to discourage self-evaluation, plausibly because both smells stem from a similar intent to limit the model's introspective behavior.

As expected given its definition, the *Low Quality* smell shows its highest correlations with the three smells from which it is derived, namely *Poor Grammar* ($r = 0.56$), *Poor Formatting* ($r = 0.53$), and *Poor Readability* ($r = 0.34$). Notably, however, its correlation with the latter is comparatively weaker, suggesting that grammar and formatting issues contribute more consistently to the perceived quality score than readability does. This divergence is further illuminated by the negative correlation between *Poor Readability* and *Complexity-Length* ($r = -0.35$), the strongest negative relationship observed in the matrix, which implies that longer or more complex prompts are not necessarily associated with poor readability.

👉 **Answer to RQ$_2$.** The analysis conducted on a dataset consisting of 42,140 real-world prompts revealed that pattern-related smells (e.g., *Role Suppression* and *Reasoning Suppression*) are considerably more prevalent than syntactic smells, and that most smells behave as largely independent dimensions of prompt quality, with the notable exception of the strong association between *Reasoning Suppression* and *Lack of Self-Reflection*. The anomalous distribution of the *Complexity-Length* smell further suggests that, while the catalog and its metrics are broadly applicable to large-scale corpora, some of the proposed thresholds may require refinement before being adopted as reliable indicators of prompt quality.

4.3 RQ3: Empirical Validation of Smells Mitigation

As stated in Research Q3, mitigating a prompt smell is only half of the problem: it must also be verified, by means of a subsequent check, whether the smell has actually been removed. Section 3.4 described the design of an automatic correction mechanism for three of the ten catalogued smells, namely *Role Suppression*, *Reasoning Suppression*, and *Lack of Self-Reflection*. This section reports the empirical validation of that mechanism when applied at scale to the dataset introduced in Section 3.3.1, in order to determine whether the corrected prompts are indeed free of the targeted smells upon re-evaluation. The remainder of the section first describes the correction and re-evaluation process and its outcomes (Section 4.3.1), then investigates whether the residual false positives are related to prompt length (Section 4.3.2).

4.3.1 Large-Scale Correction and Re-evaluation

Taking as a starting point the dataset obtained in the previous phase, we filtered it to retain only the prompts in which at least one of the *Reasoning Suppression*, *Role Suppression*, and *Lack of Self-Reflection* smells was detected. Since the latter smell alone is present in 99% of the prompts, as many as 41,906 prompts passed the filter, meaning that only 234 prompts out of 42,140 exhibited none of the three smells under consideration. Within this filtered dataset, no role is assigned to the LLM in 93.00% of the prompts, the LLM is not asked to break down its reasoning step by step in 98.24% of the prompts, and the LLM is not asked to review its generated output before returning the response in 99.04% of the prompts.

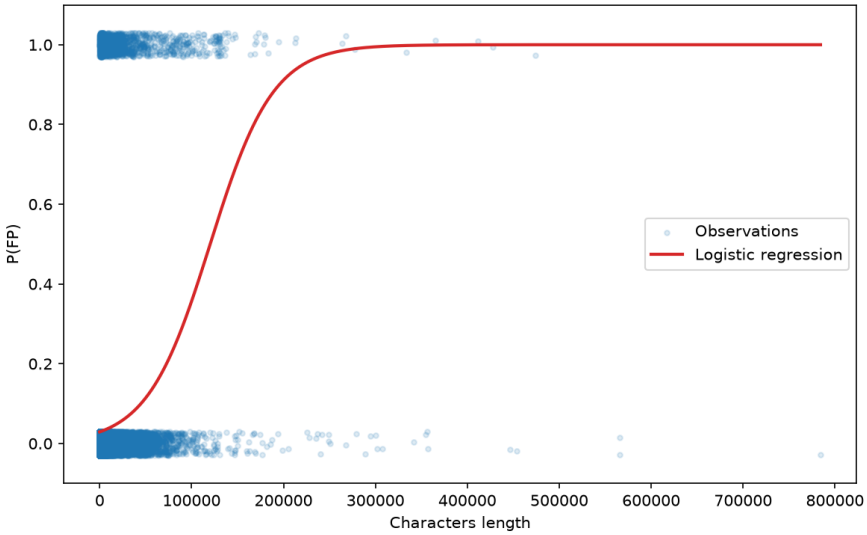
The filtered dataset was subsequently processed by the correction tool described in Section 3.4.1, which addressed whichever of the three targeted smells were present in each prompt. It was expected that, upon re-running the detection algorithm introduced in Section 3.3 on the corrected dataset, the prevalence of *Reasoning Suppression*, *Role Suppression*, and *Lack of Self-Reflection* would drop to zero. This expectation, however, was only partially confirmed: the *Role Suppression* smell was still diagnosed in 1.87% of the prompts corrected (783 prompts), the *Reasoning Suppression* smell in 3.91% of the prompts (1,638 prompts), and the *Lack of Self-Reflection* smell in 2.81% of the prompts (1,177 prompts). This phenomenon can plausibly be attributed to the intrinsic non-determinism of LLMs, which may occasionally fail to recognize the injected corrective sentences, for instance by misclassifying them as part of the original request rather than as explicit role, reasoning, or self-review instructions.

4.3.2 Relation Between False Positives and Prompt Length

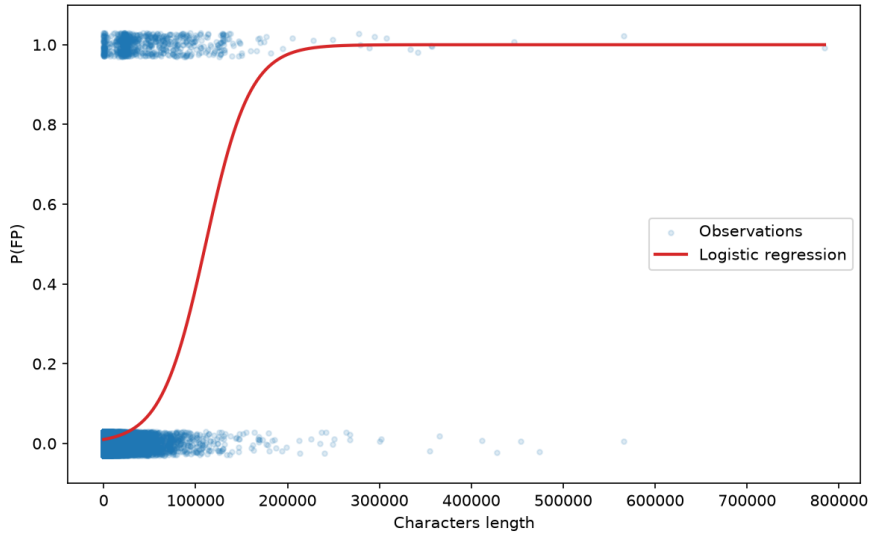
To investigate whether prompt length is related to the false positives detected on the prompts already corrected, a logistic regression model was fitted for each of the three targeted smells, using the full dataset of 41,904 corrected prompts. For each model, the character length of the prompt was used as the sole predictor, while the corresponding smell flag served as the binary target variable. Figures 4.3a, 4.3b, and 4.3c show, for each smell, the resulting logistic regression curve superimposed on the individual observations.

Figure 4.3: Estimated probability of false positives as a function of length

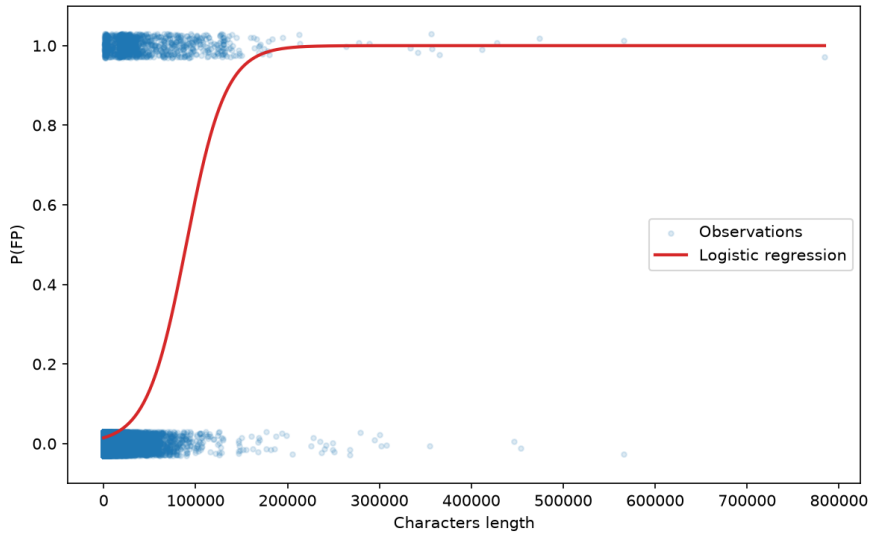
(a) Reasoning Suppression



(b) Role Suppression



(c) Lack of Self-Reflection



All three curves display a clear sigmoidal pattern, with the estimated probability of a false positive remaining close to zero for shorter prompts and rising sharply within an intermediate length range, before saturating near 1 for the longest prompts in the dataset. Specifically, both *Role Suppression* (Figure 4.3b) and *Lack of Self-Reflection* (Figure 4.3c) exhibit a steep increase roughly between 50,000 and 200,000 characters, reaching a false-positive probability close to 1 beyond that range. In the case of *Role Suppression*, this behavior is plausibly related to the fact the sentence "`Act as an experienced software engineer`" is injected at the very beginning of the prompt: as the overall prompt length grows, this instruction may become comparatively less salient to the LLM judge, which could get sidetracked while processing the remaining, much longer body of text. *Reasoning Suppression* (Figure 4.3a) follows a similar overall shape, but the transition is noticeably smoother and shifted further to the right, with the probability approaching 1 only beyond approximately 250,000-300,000 characters.

Overall, these results indicate a consistent and non-negligible relationship between the prompt length and the tendency to erroneously re-flag a smell already corrected: the longer the prompt, the more likely the judge is to produce a false positive, for all three smells alike. This effect becomes particularly pronounced among the most extreme prompts in the dataset: considering only the 420 prompts exceeding the 99th percentile of length (75,327 characters), the false-positive rate rises to 41.9% for *Role Suppression*, 42.6% for *Reasoning Suppression*, and 53.6% for *Lack of Self-Reflection*, substantially higher than the rates observed across the dataset as a whole. This suggests that, as prompts grow very long, the LLM judge becomes progressively less reliable at correctly recognizing the injected sentences, regardless of which specific smell is being assessed.

👉 **Answer to RQ$_3$.** No approach was identified for mitigating all ten prompt smells; a mitigation strategy could be devised only for three of them, namely *Reasoning Suppression*, *Role Suppression*, and *Lack of Self-Reflection*. An experiment conducted on the entire dataset revealed that, even when a smell is corrected in a deterministic manner, it may still be erroneously flagged as present by the LLM-as-a-judge-based detection algorithm.

CHAPTER 5

The Development of NasoMano

This chapter illustrates the process that led to the creation of *NasoMano*, a tool capable of detecting ten prompt smells within prompts written in English and correcting three of them, using the algorithms implemented in Chapter 3 — Reasearch Method. It consists of two independent tools: *Naso* and *Mano*. The word "naso" is the Italian equivalent of "nose", the organ of the human body capable of detecting smells, while the word "mano" is the Italian word for "hand", the part of the body which can be used to modify something. The tools have been developed in Python and can be used either individually, via a REST API, or through a minimal graphical user interfaced.

5.1 Naso - prompt smells detection tool

As said in the chapter introduction, *Naso* is capable of detecting ten prompt smells within prompts written in English: five of them are those identified through the tertiary review conducted in Chapter 3, whereas the other five were acquired from a study of Della Porta et al. [1]. Table 5.1 lists all ten smells that the *Naso* tool can detect, and their associated evaluation metrics.

Table 5.1: List of prompt smells' metrics that the tool *Naso* can compute

Prompt Smells Metrics	Metric Computation	Origin
Reasoning Suppression	True if the CoT pattern or its variants are omitted; false otherwise.	Our SLR
Lack of Self-Reflection	True if the LLM is not required to double-check its output, false otherwise.	Our SLR
Role Suppression	True if the Persona pattern is omitted, false otherwise.	Our SLR
Unspecified Output Structure	True if the format of the output structure is not specified, false otherwise.	Our SLR
Lack of Examples	The count of examples provided.	Our SLR
Grammatical Correctness Score (G)	$G = 1 - \frac{n_{\text{matches}}}{\max(1, n_{\text{words}})}$ *Where* n_{matches}: grammar / spelling issues; n_{words}: *words*.	[1]
Formatting Score (F)	Normalized combination of punctuation (40% weight), capitalization (40%) and layout indicators (20%).	[1]
Readability Score (R)	The *Flesch Reading Ease* readability metric.	[1]
Prompt Quality Score (PQS)	$PQS = \frac{G + F + C}{3}$ *Where G: grammatical correctness; F: formatting quality; C: readability.*	[1]
Complexity-Length Score (CL)	$CLS = 1 - \min\left(1, \frac{\frac{WC}{WC_{\max}} + \frac{GFI}{20}}{2}\right)$ *WC: word count;* $WC_{\max}$*: length threshold; GFI: Gunning Fog Index.*	[1]

5.1.1 Syntactic Smells Detection

Section 3.3.2 provides a high-level explanation of how smells are detected within prompts. This section, by contrast, focuses on explaining how the detection algorithm was implemented and which libraries were employed to this end.

To interface with the *LanguageTool* tool, the *language_tool_python* library was used [29]; specifically, the `check(prompt)` method was employed to detect grammatical errors.

The *Readability* score is simply the *Flesch Reading Ease* index scaled to the $[0, 1]$ range. To calculate this index, *Naso* employs the `flesch_reading_ease(prompt)` method from the *textstat* Python library [35]. The same library is used to compute the *Gunning Fog Index*, required in the formula of the *Complexity-Length* score, through the `gunning_fog(prompt)` method. Finally, the `lexicon_count(prompt)` method

from this same library was also used to obtain the word count of the prompt for the computation of the *Grammar Correctness* score and the *Complexity-Length* score.

As regards the *Formatting* score, regular expressions were employed through Python's `re` library.

5.1.2 LLM-as-a-judge Inference Configuration

The five prompt smells related to anti-patterns are detected using an LLM-as-a-judge architecture. During the experiments conducted in Section 3.3, the `qwen2.5:7b` model was chosen because it was considered a good trade-off between the number of tokens that can be processed and response quality.

Leveraging its open-weight nature, the selected model is not accessed via a remote API, but rather executed locally using Ollama [36]. Ollama is an open-source framework designed to deploy and execute large language models locally on consumer-grade hardware. The way Ollama allows developers to interact with local LLMs is via a simple REST API or, as in our case, a Python client called `ollama`, made available by the framework's developers themselves [37].

Figure 5.1: Ollama client configuration

```
client.chat(
    model="qwen2.5:7B",
    messages=[
        {"role": "system", "content":
        "You are a helpful JSON-outputting assistant."},
        {"role": "user", "content": full_prompt}
    ],
    format=PromptEvaluation.model_json_schema(),
    options={"temperature": 0.1}
)
```

Figure 5.1 illustrates the configuration of the Ollama client function call that enables the use of the model. A critical hyperparameter in this setup is the temperature, which modulates the probability distribution of the model's next-token generation (i.e., its level of creativity). By setting the temperature to an extremely low value like 0.1, the generation process is forced to return a response that is as deterministic as

possible.

Guided Decoding

To further guarantee the robustness of the automated evaluation pipeline, it was necessary to ensure that the LLM judge's response could be parsed without failure. Relying solely on natural language instructions in the prompt to enforce a JSON format is inherently error-prone. Therefore, a Pydantic model called `PromptEvaluation` was defined to implement a strategy called Guided Decoding. This technique consists in constraining the LLM's token generation space at inference time, ensuring that the output strictly adheres to a target schema. Figure 5.2 shows the definition of the Pydantic model used, which in turn is parsed into JSON using the `BaseModel.model_json_schema` function (see Figure 5.1).

Figure 5.2: Pydantic model used for guided decoding

```
1    class PromptEvaluation(BaseModel):
2        reasoning: str = Field(description="The reasoning behind your
         ↪  decisions for each dimension.")
3        is_role_assigned: bool
4        is_reasoning_required: bool
5        self_reflection_present: bool
6        structure_specified: bool
7        examples_count: int
```

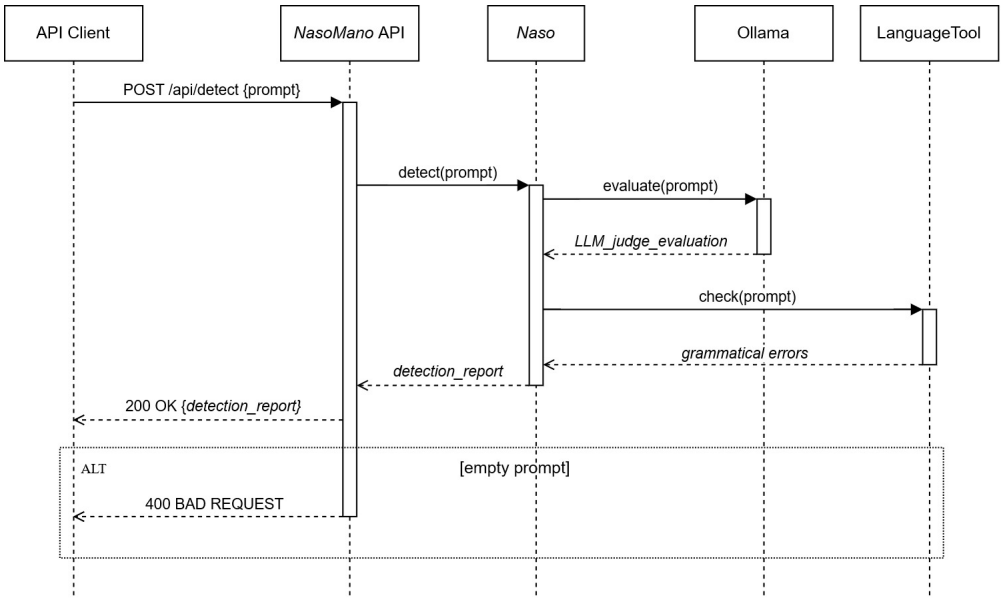
5.2 The NasoMano API

The *NasoMano* tool can be used via a REST API implemented in Python using the FastAPI library [38]. The API exposes three endpoints:

- `/api/detect` allows the *Naso* tool to be used for detecting prompt smells. To make a request to this endpoint, a `prompt` field must be included in the body. It returns a JSON-formatted dictionary containing the fields `llm_reasoning` — i.e., a string corresponding to the LLM judge's rationale behind the classification of the five anti-pattern smells —, `metrics`, and `smells_detected`.

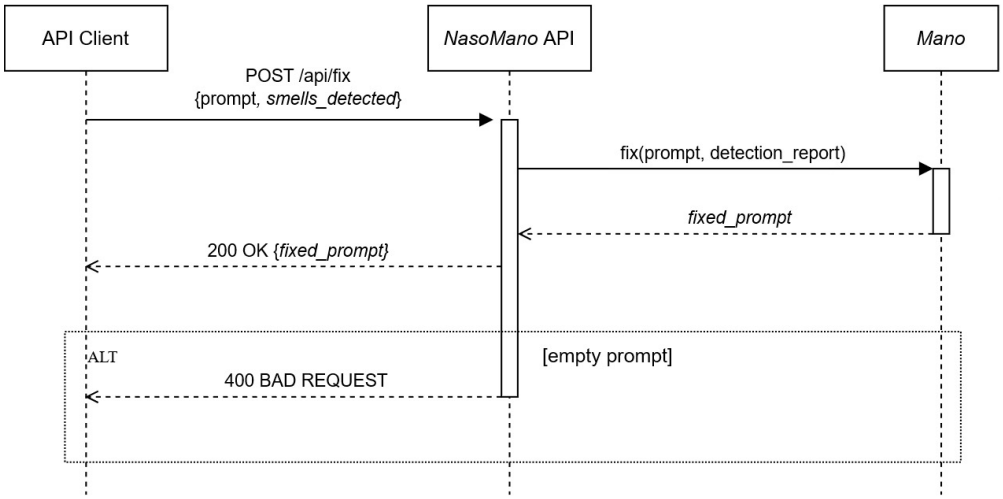
The latter two are themselves dictionaries containing, respectively, the values of the metrics associated with the prompt smells and a boolean flag indicating the presence or absence of each prompt smell. Optionally, two variants of the endpoint can be used, namely `/api/detect/syntactic` and `/api/detect/antipatterns`, to detect only the syntactic prompt smells and those associated with anti-patterns, respectively. In the resulting output dictionary, the fields not involved in the request — e.g., `llm_reasoning` and the anti-pattern smell metrics in the first case — are set to `null`.

Figure 5.3: UML sequence diagram for the `/api/detect` endpoint



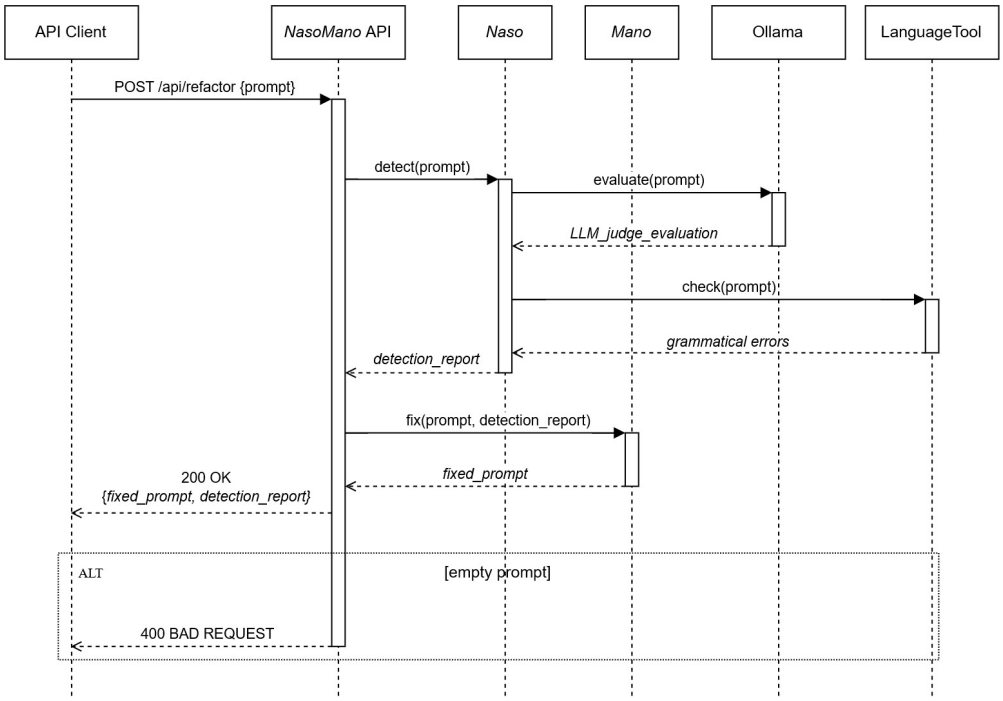
- `/api/fix`, on the other hand, allows smells to be corrected using the *Mano* tool. In addition to the `prompt` field, the request body must also include a `smells_detected` field, i.e., a key-value dictionary with three boolean values: `reasoning_suppression`, `lack_of_self_reflection`, and `role_suppression`. *Mano* will inject into the prompt only the sentences associated with the prompt smells whose values are set to `true`. The endpoint returns a string corresponding to the fixed prompt.

Figure 5.4: UML sequence diagram for the `/api/fix` endpoint



- `/api/refactor` combines the functionalities of the two tools. As usual, the `prompt` field must be included in the request body. It returns as output a JSON-formatted dictionary consisting of the fields `fixed_prompt` and `detection_report`; the latter is simply the dictionary returned as output by the `/api/detect` endpoint.

Figure 5.5: UML sequence diagram for the `/api/refactor` endpoint



5.3 NasoMano Class Diagram

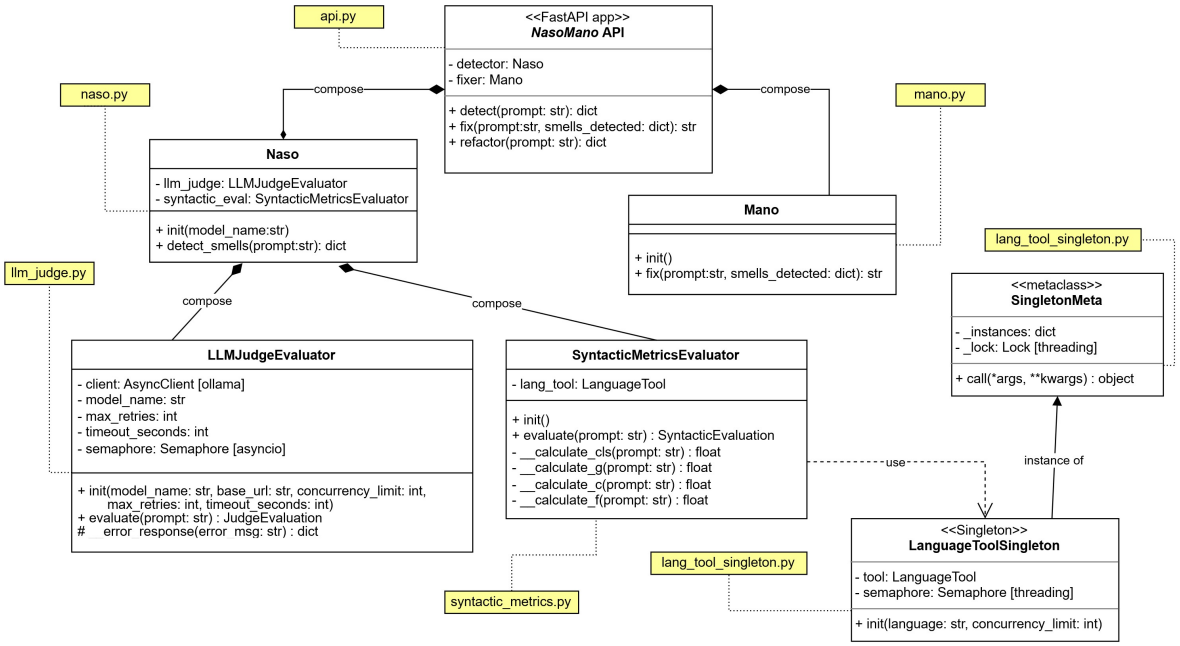


Figure 5.6: UML class diagram of the *NasoMano* tool

5.4 Containerization of the API

Once its implementation was completed, the tool was distributed as a Docker image. Docker is a platform that packages an application together with its dependencies into a container [39]. A container is an isolated process that shares the kernel of the host, instead of emulating a whole operating system as a virtual machine does, and for this reason it starts quickly and has a small footprint. Containers are created from images, which are read-only templates built by following the instructions of a `Dockerfile`. Each instruction that modifies the file system produces a layer, and an image is the stack of these layers. Layers are cached and reused across builds, but their sizes also add up, a fact that matters for the choices described below.

There are many reasons why it was decided to distribute the tool as a Docker image. First of all, containerization simplifies installation, and therefore access to the tool, since users no longer have to set up its dependencies manually, and it avoids the "works on my machine" problem. A container also keeps these dependencies separate from whatever else is installed on the host. Isolation has a further side effect

on security: a service running in a minimal environment exposes less to an attacker than one installed on a general-purpose system.

The image was built with two aims, keeping its size small and keeping the number of known vulnerabilities low. The first good practice employed was to split the build into two stages within a single `Dockerfile`, separating the build environment from the runtime one. Compilers, package caches and intermediate files remain in the first stage, which leaves the final image with fewer layers and fewer packages that a CVE could affect. The number of layers was reduced further by grouping `RUN` commands with `&&`, since every `RUN` instruction creates a new layer. On the subject of vulnerabilities, a second measure was to avoid running the service as root. By default, a process inside a container has root privileges, so a flaw in the application would hand an attacker far more than it should. The image therefore defines a dedicated system user with no login shell. The source files remain owned by root and are read-only for this user, so a compromised process cannot modify the code it is running.

Once built, the image, which weighs 284.33 MB, was uploaded to Docker Hub at `https://hub.docker.com/r/lucasalvaa/nasomano-api`, so that anyone wishing to use the tool can download it easily. In the working repository was also included a `docker-compose.yml` file that defines, besides the API, the Ollama and LanguageTool services on which the tool depends. Running them as separate containers spares the user from installing and configuring each of them by hand, and it keeps the API image small, since neither a Java runtime nor a model server has to be bundled into it. The whole system starts with a single command, `docker compose up`, who can also download the images from Docker Hub if they are not already on the device.

5.5 Graphical User Interface

A minimal graphical user interface was developed using HTML, CSS, and JavaScript to make the tool more accessible to end users. Upon opening the page, the user is presented with a simple input bar in which to type the desired prompt. Once the user presses enter, the graphical client sends an HTTP request to the `/api/refactor`

endpoint of the API. Upon receiving a response from the server, the interface displays a box containing the list of the ten prompt smells, with each label colored green or red depending on whether the corresponding smell was detected, so as to ensure a more immediate understanding for the end user. Furthermore, if at least one of the three prompt smells correctable by *Mano* has been detected, the box also displays the corrected prompt along with a set of recommended fixes for resolving the remaining prompt smells not fixed by the tool, e.g., `You should add some examples to clarify the context or the expected output`. A button is also provided to copy the corrected prompt to the clipboard, further enhancing the user-friendliness of the site.

CHAPTER 6

Conclusions

6.1 Summary of the Thesis

Nowadays, an increasing number of people rely on Large Language Models on a daily basis to generate or refine text, code, and e-mails, or simply to ask questions. A natural consequence of the rapid diffusion of these models has been the emergence of Prompt Engineering, a research field concerned with identifying novel techniques that, when applied by the user at the time of prompt formulation, can improve the quality of an LLM's inherently non-deterministic response. However, very few researchers have focused on treating the prompt itself as an artifact whose quality can be assessed independently of the output it elicits.

In 2024, Ronanki et al. [10] were the first to introduce a formal definition of *prompt smells* in the literature, drawing an explicit parallel with the notion of *code smells* in software engineering. In early 2026, Della Porta et al. [1] compiled an initial catalog of eight prompt smells, organized into two categories, namely semantic smells and syntactic smells. This catalog, however, exhibited three notable limitations: it was not derived systematically, having been compiled through a rapid rather than a systematic literature review; the identified smells were not empirically evaluated on a substantial volume of data; and, finally, the authors left open the question of

whether these smells could actually be mitigated.

This thesis originates from the intent to contribute to research on prompt smells by addressing these three gaps. To that end, a research question was formulated for each of the three gaps.

To address the first gap, i.e., the systematic compilation of a prompt smells catalog together with its associated metrics, a tertiary study was conducted on systematic literature reviews concerning prompt engineering within software development processes. To address the second one, an LLM-as-a-judge-based algorithm was implemented to measure the prevalence of the resulting smells across a large-scale dataset of real-world developer-LLM prompts, and a statistical analysis was conducted to investigate their distribution and mutual correlations. Finally, to address the third gap, concerning the possibility of mitigating prompt smells, a correction strategy was devised and validated for a subset of the cataloged smells, by injecting into the prompt sentences compliant with the corresponding prompt pattern and subsequently re-running the detection algorithm on the corrected prompts.

Following the completion of the research, the detection and correction algorithms employed throughout the study were encapsulated within an open-source tool named *NasoMano*. The tool is accessible both through a REST API and through a minimal Graphical User Interface. The API allows *Naso* and *Mano*, namely the detection and correction components, respectively, to be invoked independently of one another.

6.2 Main Findings

- **Research Question 1**. The tertiary study conducted to address the first gap identified eleven recurring bad practices, although only five of them could be associated with a corresponding metric, namely *Reasoning Suppression*, *Lack of Self-Reflection*, *Role Suppression*, *Unspecified Output Structure*, and *Lack of Examples*. All five of these prompt smells can be regarded as anti-patterns, in that each corresponds to the omission of a prompt pattern already established in the literature — e.g., *Role Suppression* corresponds to the non-adoption of the *Persona* pattern.

- **Research Question 2**. The large-scale measurement conducted to address the second gap was performed on a dataset comprising more than 42,000 real-world, English-language developer-LLM prompts, covering both the five anti-pattern smells identified through the tertiary study and the five smells belonging to the catalog proposed by Della Porta et al. This experiment revealed that four out of the five syntactic smells are present in more than 58% of the dataset, with the notable exception of the *Complexity-Length* smell, whose prevalence amounts to only 7.1%; the reasons underlying this markedly lower prevalence warrant further investigation. In turn, four out of the five anti-pattern smells were detected in more than 92% of the dataset, with the exception of *Unspecified Output Structure*, whose prevalence amounts to "only" 79.3%. With respect to the correlations among the different prompt smells, only *Reasoning Suppression* and *Lack of Self-Reflection* exhibited a particularly strong association ($r = 0.50$).

- **Research Question 3**. A correction strategy was devised for only three of the cataloged smells, namely *Reasoning Suppression*, *Role Suppression*, and *Lack of Self-Reflection*, by injecting into the prompt sentences compliant with the corresponding prompt pattern. This correction algorithm was applied to the entirety of the aforementioned dataset, after which the detection algorithm was re-run on the corrected prompts. Somewhat unsurprisingly, it emerged that, as prompt length increases, the LLM-based detection tool — and therefore plausibly LLMs in general — struggles to correctly recognize the injected prompt patterns. Additionally, an attempt was made to mitigate the syntactic smells by means of the `correct()` function provided by the *LanguageTool* library; however, a considerable number of erroneous corrections were observed even on short pieces of text, leading to the abandonment of this approach.

6.3 Future Developments

- Both the systematic literature review conducted in this study and the pre-existing catalog proposed by Della Porta et al. include smells of a qualitative nature for which no measurement approach has been identified. Examples of

such smells include the presence of bias, an excessively detailed prompt and the total absence of context. Measuring these smells proves non-trivial, since, unlike patterns such as *Reasoning* and *Persona/Role*, whose absence can be recognized relatively easily even by a human reader, they may remain challenging even upon manual inspection. For example, it would be necessary to empirically determine a boundary that establishes when the context of the request is not detailed enough, detailed enough, or too detailed.

- As many as seven smells can be detected by the *Naso* tool but cannot be corrected by the *Mano* tool. For two of them, namely *Lack of Examples* and *Unspecified Output Structure*, it was decided to forgo correction altogether, since addressing them would require LLM-based techniques; given that even human users may introduce bias when adding examples to a prompt, it seems preferable not to have such examples generated by an LLM, which would in any case first need to fully grasp the context of the request. For the syntactic smells, a rapid approach relying on the `LanguageTool.correct()` function was instead attempted, yielding disappointing results. A more in-depth knowledge of this or other tools could help identify new solutions for addressing these errors.

- The measurements obtained for the *Complexity-Length* score, one of the smells identified earlier, proved to be considerably anomalous with respect to the remaining nine smells. Further investigation is warranted to determine whether these measurements are indeed accurate; possible directions include conducting new white-box measurements on individual prompts to assess whether the underlying formula should be revised, for instance by increasing the *length threshold*, or examining whether this evaluation is affected by the presence of code snippets within prompts, which are plausibly detected as plain, non-complex English text.

Bibliography

[1] A. Della Porta, G. Voria, A. Abbate, R. Sulipano, S. Lambiase, G. Catolino, and F. Palomba, "Toward Measuring Prompt Quality: A Preliminary Investigation on Prompt Smells," in *2026 IEEE International Conference on Software Analysis, Evolution and Reengineering - Companion (SANER-C)*, 2026, pp. 293–300. (Citato alle pagine 2, 10, 11, 15, 32, 46, 47 e 55)

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," 2023. [Online]. Available: https://arxiv.org/abs/1706.03762 (Citato a pagina 7)

[3] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating Large Language Models Trained on Code," 2021. [Online]. Available: https://arxiv.org/abs/2107.03374 (Citato a pagina 8)

[4] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve, "Code Llama: Open Foundation Models for Code," 2024. [Online]. Available: https://arxiv.org/abs/2308.12950 (Citato a pagina 8)

[5] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," 2023. [Online]. Available: https://arxiv.org/abs/2201.11903 (Citato a pagina 9)

[6] J. Li, G. Li, Y. Li, and Z. Jin, "Structured Chain-of-Thought Prompting for Code Generation," 2023. [Online]. Available: https://arxiv.org/abs/2305.06599 (Citato a pagina 9)

[7] X. Zhao, M. Li, W. Lu, C. Weber, J. H. Lee, K. Chu, and S. Wermter, "Enhancing Zero-Shot Chain-of-Thought Reasoning in Large Language Models through Logic," 2024. [Online]. Available: https://arxiv.org/abs/2309.13339 (Citato a pagina 9)

[8] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, and D. C. Schmidt, "A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT," 2023. [Online]. Available: https://arxiv.org/abs/2302.11382 (Citato a pagina 9)

[9] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language Models are Few-Shot Learners," 2020. [Online]. Available: https://arxiv.org/abs/2005.14165 (Citato a pagina 10)

[10] K. Ronanki, B. Cabrero-Daniel, and C. Berger, "Prompt Smells: An Omen for Undesirable Generative AI Outputs," 2024. [Online]. Available: https://arxiv.org/abs/2401.12611 (Citato alle pagine 10 e 55)

[11] B. Kitchenham and S. Charters, "Guidelines for performing Systematic Literature Reviews in Software Engineering," vol. 2, 01 2007. (Citato a pagina 16)

[12] C. Wohlin, "Guidelines for snowballing in systematic literature studies and a replication in software engineering," in *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE '14. New York, NY, USA: Association for Computing Machinery, 2014. [Online]. Available: https://doi.org/10.1145/2601248.2601268 (Citato a pagina 16)

[13] R. Singh, P. Singh, A. Malik, and D. Sukmawan, "Understanding and Mitigating Hallucinations in Large Language Models: Insights from a Systematic Literature Review," 04 2025, pp. 1–10. (Citato a pagina 21)

[14] K. Huang, F. Wang, Y. Huang, and C. Arora, " Prompt Engineering for Requirements Engineering: A Literature Review and Roadmap ," in *2025 IEEE 33rd International Requirements Engineering Conference Workshops (REW)*. Los Alamitos, CA, USA: IEEE Computer Society, 09 2025, pp. 548–557. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/REW66121.2025.00081 (Citato a pagina 21)

[15] Y. Gutierrez Sarria, E. Camacho, C. Pardo, and V. Villarreal, "Prompts Engineering Challenges in Software Code Generation," 09 2025. (Citato a pagina 21)

[16] K. Ronanki, S. Arvidsson, and J. Axell, "Prompt Engineering Guidelines for Using Large Language Models in Requirements Engineering," 07 2025. (Citato alle pagine 21 e 38)

[17] Y.-Y. Liu, Z. Zheng, F. Zhang, J.-C. Feng, Y.-Y. Fu, J.-D. Zhai, B.-S. He, X. Zhang, and X.-Y. Du, "A comprehensive taxonomy of prompt engineering techniques for large language models," *Front. Comput. Sci.*, vol. 20, no. 3, Oct. 2025. [Online]. Available: https://doi.org/10.1007/s11704-025-50058-z (Citato alle pagine 21 e 38)

[18] Y. Sasaki, H. Washizaki, J. Li, N. Yoshioka, N. Ubayashi, and Y. Fukazawa, "Landscape and Taxonomy of Prompt Engineering Patterns in Software Engineering," *IT Professional*, vol. 27, no. 1, pp. 41–49, 2025. (Citato a pagina 21)

[19] C. Tony, N. E. Díaz Ferreyra, M. Mutas, S. Dhif, and R. Scandariato, "Prompting Techniques for Secure Code Generation: A Systematic Investigation," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 8, Oct. 2025. [Online]. Available: https://doi.org/10.1145/3722108 (Citato alle pagine 21 e 39)

[20] I. W. Syahputri, E. K. Budiardjo, and P. O. H. Putra, "Unlocking the Potential of the Prompt Engineering Paradigm in Software Engineering: A Systematic Literature Review," *AI*, vol. 6, no. 9, 2025. [Online]. Available: https://www.mdpi.com/2673-2688/6/9/206 (Citato alle pagine 21 e 38)

[21] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, "A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications," 2025. [Online]. Available: https://arxiv.org/abs/2402.07927 (Citato alle pagine 21 e 38)

[22] K. Yang, J. Liu, J. Wu, C. Yang, Y. R. Fung, S. Li, Z. Huang, X. Cao, X. Wang, Y. Wang, H. Ji, and C. Zhai, "If LLM Is the Wizard, Then Code Is the Wand: A Survey on How Code Empowers Large Language Models to Serve as Intelligent Agents," 2024. [Online]. Available: https://arxiv.org/abs/2401.00812 (Citato a pagina 21)

[23] F. A. Pinto, "From Prompt Quality to Code Quality: An Empirical Study on Developer–LLM Conversations," Master's thesis, Università degli Studi di Salerno, Fisciano, Salerno, Italy, 2026. (Citato a pagina 22)

[24] W. Zhao, X. Ren, J. Hessel, C. Cardie, Y. Choi, and Y. Deng, "WildChat: 1M ChatGPT Interaction Logs in the Wild," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=Bl8u7ZRlbM (Citato a pagina 22)

[25] S. Zhong, Y. Zou, and B. Adams, "Developer-LLM Conversations: An Empirical Study of Interactions and Generated Code Quality," 2025. [Online]. Available: https://arxiv.org/abs/2509.10402 (Citato a pagina 23)

[26] A. Della Porta, S. Lambiase, V. Pontillo, J. F. Ferreira, A. A. Rahman, C. Ragkhitwetsagul, and F. Palomba, "Understanding Prompt Quality and Its Relation to LLM-based Code Generation," *IEEE Transactions on Software Engineering*, pp. 1–20, 2026. (Citato a pagina 24)

[27] R. Gunning, *The Technique of Clear Writing*. New York: McGraw-Hill, 1952. (Citato a pagina 25)

[28] R. Flesch, "A new readability yardstick," *The Journal of applied psychology*, vol. 32, no. 3, pp. 221–233, 1948. (Citato a pagina 25)

[29] jxmorris12 and contributors, "language_tool_python: a free, non-AI python grammar checker," https://https://github.com/jxmorris12/language_tool_python, 2026. (Citato alle pagine 25 e 47)

[30] D. Naber, "A Rule-Based Style and Grammar Checker," Master's thesis, University of Bielefeld, Bielefeld, Germany, 2003. [Online]. Available: https://languagetool.org (Citato a pagina 25)

[31] Q. Team, "Qwen2.5: A Party of Foundation Models," September 2024. [Online]. Available: https://qwenlm.github.io/blog/qwen2.5/ (Citato a pagina 28)

[32] J. Cohen, "A Coefficient of Agreement for Nominal Scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960. (Citato a pagina 30)

[33] J. R. Landis and G. G. Koch, "The Measurement of Observer Agreement for Categorical Data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977. (Citato a pagina 30)

[34] A. Efrat and O. Levy, "The Turking Test: Can Language Models Understand Instructions?" *ArXiv*, vol. abs/2010.11982, 2020. [Online]. Available: https://api.semanticscholar.org/CorpusID:225062157 (Citato a pagina 38)

[35] A. Ward and contributors, "textstat: a Python package to calculate readability statistics of a text object," https://github.com/textstat/textstat, 2026, version 0.7.13. (Citato a pagina 47)

[36] Ollama Team, "Ollama," https://github.com/ollama/ollama. [Online]. Available: https://github.com/ollama/ollama (Citato a pagina 48)

[37] ——, "Ollama Python Library," https://github.com/ollama/ollama-python. [Online]. Available: https://github.com/ollama/ollama-python (Citato a pagina 48)

[38] S. Ramírez and contributors, "FastAPI," https://github.com/fastapi/fastapi. [Online]. Available: https://github.com/fastapi/fastapi (Citato a pagina 49)

[39] D. Merkel, "Docker: lightweight Linux containers for consistent development and deployment," *Linux J.*, vol. 2014, no. 239, Mar. 2014. (Citato a pagina 52)

Ringraziamenti

Il primo, sentito ringraziamento va al professor Palomba: la sua grande professionalità e la passione con cui insegna mi hanno incantato fin da subito e sono una fonte di ispirazione. Ho apprezzato altresì la sua tempestiva disponibilità in questi mesi di tirocinio e di stesura della tesi: chi legge mi creda, l'aggettivo «tempestiva» non è lì tanto per riempire la frase. È stato un vero onore essere suo tesista.

Un doveroso ringraziamento va ad Antonio e Gianmario, che mi hanno seguito con grande attenzione lungo tutto il percorso. Dopo ogni vostra indicazione ho avuto la sensazione che questo lavoro di tesi facesse un salto di qualità. Vi auguro il meglio per il vostro futuro e che non perdiate mai la vostra professionalità.

Infine, un grazie va ai parenti e agli amici che, nel corso di questi tre anni, hanno riposto nei miei confronti la loro fiducia e la loro stima. A chi ha condiviso con me un tratto di questo cammino devo qualcosa di più di un semplice grazie: devo il ricordo di un bel momento, di un'emozione vissuta insieme, oppure di una semplice chiacchierata. Sono cose piccole, ma sono quelle che restano e che, col tempo, si rivelano le più preziose.



This thesis has contributed to planting a tree in Tanzania through the Treedom project.

https://www.treedom.net/it/user/sesalab/event/sesa-random-forest